

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Inteligência Artificial para Sistemas Autónomos

Relatório: Projeto (Entrega Final)

Turma: 42D

Nome: Fábio Pestana Número: 50756

Professor: Luís Morgado

Data: 13 de Junho de 2024

Índice

Lista de figuras III
Lista de tabelas IV
1. Introdução 1
2. Enquadramento teórico 2
2.1 - Inteligência Artificial 2
2.2 - Engenharia de *Software* 3
2.3 - Agentes Reativos 5
2.4 - Raciocínio Automático 7
2.5 - Procura em Espaço de Estados 9
2.5.1 - Procura em Profundidade 11
2.5.2 - Procura em Grafos 11
2.5.2.1 - Procura em Largura 11
2.5.2.2 - Procura Melhor-Primeiro 12
2.5.2.3 - Procura de Custo Uniforme 12
2.5.2.4 - Procura Sôfrega 13
2.5.2.5 - Procura A* 13
2.6 - Agentes Deliberativos 13
2.7 - Processos de Decisão Sequencial 15
3. Projecto - Parte 1 17
3.1 - Modelo do problema 17
3.2 - Desenvolvimento e implementação do sistema 17
3.3 - Teste da aplicação 21
4. Projecto - Parte 2 22
4.1 - Modelo do problema 22
4.2 - Desenvolvimento e implementação do sistema 22
4.3 - Teste da aplicação 28
5. Projecto - Parte 3 29
5.1 - Modelo do problema 29
5.2 - Desenvolvimento e implementação do sistema 29
5.3 - Teste da aplicação 38
6. Projecto - Parte 4 39
6.1 - Modelo do problema 39
6.2 - Desenvolvimento e implementação do sistema 39

6.3 - Teste da aplicação 47
7. Revisão do projecto realizado 48
8. Conclusão 49
Bibliografia 50

Lista de figuras

Figura 1 - Morgado, Luís. (2024). Representação dinâmica de um sistema 4
Figura 2 - Morgado, Luís. (2024). Representação dinâmica de um sistema, com a simbologia referida. . 5
Figura 3 - Morgado, Luís. (2024). Esquema de um comportamento simples (esquerda) e um composto (direita). 6
Figura 4 - Morgado, Luís. (2024). Esquema dos mecanismos de seleção de ação, Hierarquia, Prioridade e Combinação 6
Figura 5 - Morgado, Luís. (2024). Arquitetura de um agente reativo com memória 7
Figura 6 - Morgado, Luís. (2024). Exemplo de um espaço de estados representado sob a forma de um grafo 8
Figura 7 - Morgado, Luís. (2024). Exemplo de uma árvore de procura 9
Figura 8 - Morgado, Luís. (2024). Arquitetura de um agente deliberativo 14
Figura 9 - Morgado, Luís. (2024). Planeamento automático PEE 15
Figura 10 - Morgado, Luís. (2024). Planeamento automático PDM 16
Figura 11 - Morgado, Luís. (2024). Modelo Conceptual da relação entre o agente e o ambiente 17
Figura 12 - Morgado, Luís. (2024). Tabelas que representam função de transformação do sistema (δ e λ) 21
Figura 13 - Teste do jogo 21
Figura 14 - Morgado, Luís. (2024). Esquema de um agente com um controlo reativo 24
Figura 15 - Morgado, Luís. (2024). Comportamento composto AproximarAlvo 26
Figura 16 - Morgado, Luís. (2024). Comportamento Recolher 28
Figura 17 - Teste do Sistema 28
Figura 18 - Teste da procura em profundidade 38
Figura 19 - Teste da procura sôfrega 39
Figura 20 - Morgado, Luís. (2024). Fórmula utilizada no método util_accao() 45
Figura 21 - Teste do Sistema (PEE esquerda, PDM direita) 47

Lista de tabelas

Tabela 1 - Chaves (input) e valores (evento do jogo) do dicionário 19

1. Introdução

Este projeto, foi realizado no decorrer das aulas da disciplina Inteligência Artificial para Sistemas Autónomos, tem como objetivo os alunos conseguirem demonstrar o seu domínio a nível das aprendizagens no decorrer das aulas através do código feito (baseado na arquitetura de apoio ao projeto fornecida pelos docentes da cadeira) e sua respetiva documentação.

Este projeto é dividido em 4 partes sendo que estas foram realizadas durante o semestre com o avanço da matéria. Este relatório aborda as quatro partes respetivamente, a primeira parte aborda o tema de Engenharia de *Software*, onde se utilizou a linguagem de programação orientada a objetos, Java, devido a sua facilidade de uso e recursos que facilita a compreensão de tópicos da engenharia de *software* com maior facilidade. A segunda parte consiste num tópico de Inteligência Artificial sendo este a arquitetura de um agente reativo, para a realização desta parte do projeto e das restantes utiliza-se a linguagem de programação Python. A terceira e quarta parte baseiam-se em algoritmos de inteligência artificial, estes algoritmos baseiam-se em mecanismos de procura em representações de um problema e processos de tomada de decisão por parte do agente. Nesta terceira e quarta parte o agente deixou de ser reativo e passou a ser deliberativo.

O objetivo deste relatório é examinar o desenvolvimento das aplicações feitas em aula, sendo que ambas tem como objetivo a simulação de um agente num espaço virtual, onde na primeira parte o agente apenas procura e regista o encontro com alvos e nas restantes partes o agente autónomo recolhe alvos e desvia-se de obstáculos.

2. Enquadramento teórico

2.1 - Inteligência Artificial

Inteligência Artificial consiste numa área cientifica que estuda o desenvolvimento e análise de sistemas computacionais, como agentes que agem de uma forma inteligente. Para atingir estes objetivos é necessário desenvolver modelos e teorias sobre o comportamento inteligente para assim implementar esses modelos em sistemas computacionais.

A inteligência artificial apresenta 3 paradigmas principais, sendo estes:

- **Simbólico** - onde a inteligência resulta da ação de processos computacionais sobre estruturas simbólicas (elementos discretos que descrevem informação e representam conceitos que envolvem o problema que se deseja resolver).
- **Conexionista** - inspirado pela biologia do sistema nervoso, a inteligência resulta numa rede de neurónios (unidades de processamento) relacionados entre si, sendo a informação mantida e processada nas ligações entre estas unidades.
- **Comportamental** - a inteligência neste paradigma surge de interações entre estímulos e respostas, onde o processamento da informação é baseado nestas relações, modeladas como reações e comportamentos em diferentes níveis de organização.

Um sistema autónomo inteligente, é aquele que se baseia num ciclo infinito de perceção, processamento e ação para assim o sistema conseguir percecionar o ambiente, processar a informação que recebeu e tomar ações segundo a informação recebida, com o intuito de atingir o objetivo do sistema. Este ciclo é uma característica chave que permite um sistema operar sozinho e adaptar o seu comportamento segundo as condições do ambiente que o rodeiam. Uma representação computacional deste sistema designa-se agente inteligente, sendo que este apresenta autonomia, reatividade, pró-atividade e sociabilidade, estas características dependem da finalidade do agente.

Agentes inteligentes também são sistemas capazes de aprender e racionalizar (dois conceitos de inteligência), ou seja o agente segundo as perceções do ambiente deve ser capaz de utilizar, processar e guardar a informação, para assim conseguir decidir a ação a executar que permite-lhe obter o melhor desempenho perante os seus objetivos e conhecimento disponível sobre o ambiente.

A arquitetura de um agente pode ser categorizada segundo o seu modelo, os tipos de modelos de uma arquitetura de um agente são:

- **Modelo Reativo** - Baseia-se no paradigma comportamental, onde o comportamento do agente depende das associações entre estímulos (provenientes das perceções do agente) e respostas (ações que o agente vai executar).
- **Modelo Deliberativo** - Baseia-se no paradigma simbólico, onde o agente mantêm representações (simbólicas) internas do ambiente e realiza processos de raciocínio para

tomar decisões.

- **Modelo Híbrido** - O comportamento do agente consiste numa junção dos modelos reativos e deliberativos.

2.2 - Engenharia de *Software*

Engenharia de *Software* é composta por duas palavras, software que consiste num código executável que resolve algum problema e serve algum propósito computacional. E engenharia que consiste no desenvolvimento de algo, utilizando princípios e métodos científicos.

Segundo a IEEE (*Institute of Electrical and Electronic Engineers*) engenharia de software pode ser definida como a aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software.

Os dois principais problemas na criação e manutenção de software são a sua complexidade e a mudança, pois dependendo do problema que se deseja resolver existe uma grande dificuldade em criar e implementar um sistema, com o tempo e com a evolução no mundo da informática, pode ser necessário implementar novas funcionalidades ou alterar os métodos e tecnologias usadas, caso a complexidade do sistema não tenha sido controlada este trabalho pode se tornar bastante complicado.

Complexidade é definida como o nível de dificuldade em conceptualizar um sistema e todas as suas partes, compreender suas interações e prever o seu comportamento em diferentes condições. Gerir a complexidade de um sistema é essencial para o sucesso de um projeto, pois sistemas com muita informação (para a sua descrição) muito mais complexos vão ser, o que leva a estes serem difíceis de entender, modificar e adicionar novas funcionalidades, se a complexidade não for controlada leva a um esforço de desenvolvimento muito elevado e uma grande dificuldade em escalar o projeto.

Para diminuir o problema da complexidade desorganizada utiliza-se arquitetura de software que se baseia em dois conceitos, abstração e modularização, a abstração é utilizada para criar um modelo simples com as características comuns de um sistema, onde se dá ênfase nas suas características essenciais, ignorando o resto. A modularização consiste em organizar de uma forma coesa várias partes que formam um sistema, decompondo-o em partes coerentes (pode-se aplicar factorização para eliminar redundância no sistema e garantir consistência no mesmo), para reduzir dependências entre elementos do sistema, isola-se os detalhes internos de cada parte em relação ao exterior. Com estes dois conceitos tem-se como objetivo atingir um nível de complexidade organizada.

Para definir-se um sistema segundo a qualidade da sua arquitetura pode-se usar vários indicadores:

- **Acoplamento** - Baseia-se no grau de dependência entre diferentes partes de um sistema, o objetivo de um programador de software deve ser criar um sistema com acoplamento

baixo, isso ajuda a diminuir a sua complexidade e facilita futuras mudanças.

- **Coesão** - Baseia-se em criar uma coerência entre os elementos de um subsistema, para assim existir uma maior facilidade na compreensão do mesmo, o que permite uma maior facilidade para fazer mudanças e reutilizar partes do sistema. Quanto maior o nível de coesão, menor o nível de complexidade do sistema.
- **Simplicidade** - Consiste no grau de facilidade de entender a arquitetura do sistema.
- **Adaptabilidade** - Consiste na facilidade de alterar a arquitetura para adicionar novas funcionalidades ou para alterar as já existentes.

Com estes conceitos mencionados anteriormente é possível desenvolver a capacidade de desenvolver software de uma forma organizada, isto deve ser um processo iterativo para assim utilizarmos a modularização para conseguirmos ter uma controlo sobre a complexidade do nosso projeto. Inicialmente deve-se criar um modelo conceptual do sistema, depois a arquitetura que irá permitir implementar o sistema (modelo orientado a ser implementado) e por fim a implementação, que consiste na implementação estrutural (código estrutural) e o comportamental (código que envolve o comportamento do sistema).

A modelagem de um sistema computacional é essencial para a compreensão e projeção de sistemas complexos. A modelação de um sistema baseia-se na sua dinâmica e seu comportamento.

A dinâmica consiste nos diferentes estados que o sistema pode assumir e como estes evoluem ao longo do tempo para produzir o comportamento do sistema. Esta pode ser expressada como a função de transformação, que segundo o estado do sistema e as suas entradas atuais é capaz de produzir o estado e saídas seguintes.

O comportamento do sistema consiste na expressão de interação entre as suas entradas (informação externa) e seu estado interno, resultando na ativação das saídas (gera-se informação de saída), ou seja o comportamento transforma o estado do sistema consoante o estímulo (entrada) que recebe.

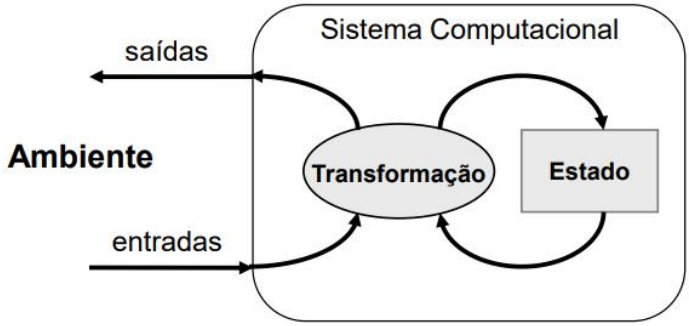


Figura 1 - Morgado, Luís. (2024). Representação dinâmica de um sistema

No modelo formal de computação de um sistema, designa-se por alfabeto o conjunto de símbolos que representa os estados possíveis do sistema, o alfabeto de entrada é designado por $\sum$ e o de saída por **Z**, o conjunto de estados internos possíveis do sistema é representado por **Q**, a função de

transformação do sistema consiste em duas funções, a função de transição de estado (representada por **δ**), que recebe uma entrada e o estado interno do sistema, o que resulta no próximo estado do sistema $\delta = Q \times \Sigma \rightarrow Q$, e a função de saída (representada por **λ**) que recebe uma entrada e um estado, resultando numa saída $\lambda = Q \times \Sigma \rightarrow Z$.

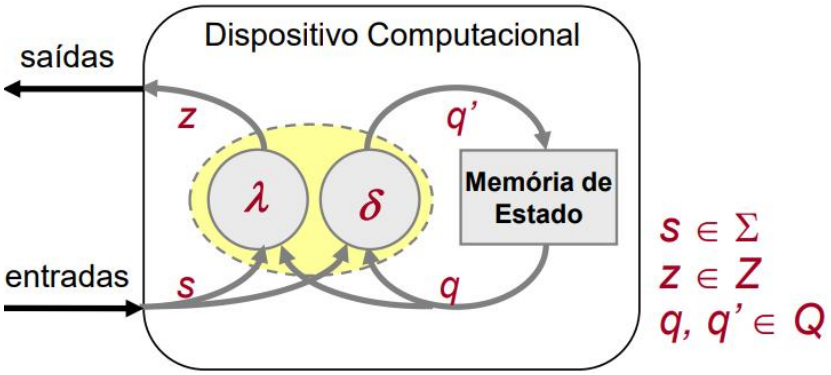


Figura 2 - Morgado, Luís. (2024). Representação dinâmica de um sistema, com a simbologia referida.

Este tipo de modelos de computação descrevem uma Máquina de estados, um mecanismo computacional que pode representar um sistema, devido ao limite de memória de um computador é necessário existir um número finito de estados. Nestes modelos existe duas versões da função de saída de um sistema (**λ**):

- Máquinas de Mealy - A função de saída depende da entrada $\lambda = Q \times \Sigma \rightarrow Z$.
- Máquinas de Moore - A função de saída não precisa da entrada $\lambda = Q \rightarrow Z$.

2.3 - Agentes Reativos

Tal como referido anteriormente no tópico 2.1 - Inteligência Artificial, numa arquitetura de um agente reativo o seu comportamento baseasse em associações entre estímulos (provenientes das perceções do agente) e respostas (ações que o agente vai executar), sendo as decisões deste agente baseadas em regras estímulo-resposta, ou seja para cada estímulo já está definida o tipo de resposta que ele deve executar. Como estes agentes apenas reagem segundo a perceção que eles tem do ambiente, significa que eles só "pensam" no instante atual, ignorando consequências que as suas ações podem ter com o tempo, isto também significa que estes agentes não necessitam de representações internas do mundo e caso aja alterações no ambiente estes conseguem ter respostas rápidas ás alterações.

Os módulos que relacionam padrões de perceção com padrões de ação, para assim atingir um objetivo são designados por comportamentos. Os comportamentos podem ser simples (reações, apenas uma associação estímulo-resposta) ou compostos (comportamentos que reúnem outros comportamentos simples ou compostos que estão relacionados entre si).

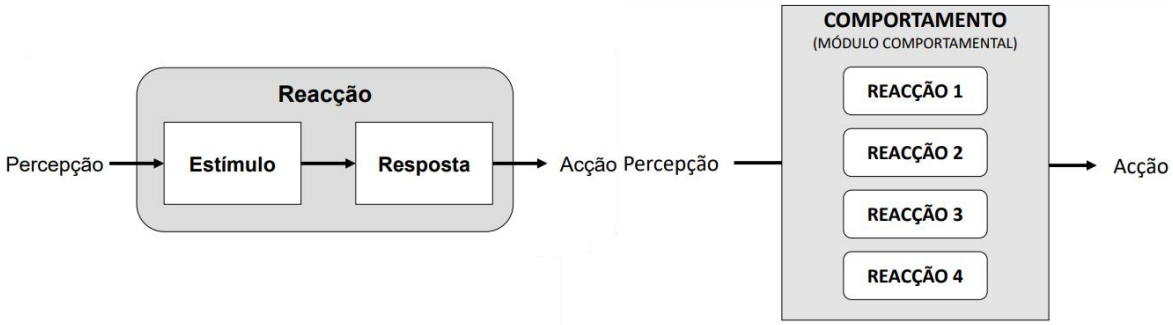


Figura 3 - Morgado, Luís. (2024). Esquema de um comportamento simples (esquerda) e um composto (direita).

Como um comportamento, pode ter várias reações, sendo que um estímulo pode ativá-las, é necessário determinar a ação a executar em função das várias respostas dos comportamentos internos, para isto usa-se um mecanismo de seleção de uma ação. Três abordagens comuns destes mecanismos são:

- **Hierarquia** - organização dos comportamentos numa hierarquia fixa que suprime e substitui, ou seja a ação que resulta da reação de mais alto nível da hierarquia, é aquela que o agente executa.
- **Prioridade** - Cada ação tem uma prioridade associada, sendo este um valor dinâmico que varia segundo as perceções do agente, a ação selecionada neste mecanismo será aquela que possui uma prioridade superior.
- **Combinação** - neste mecanismos combina-se as respostas das reações numa resposta composta, para assim obter apenas uma ação a executar pelo agente.

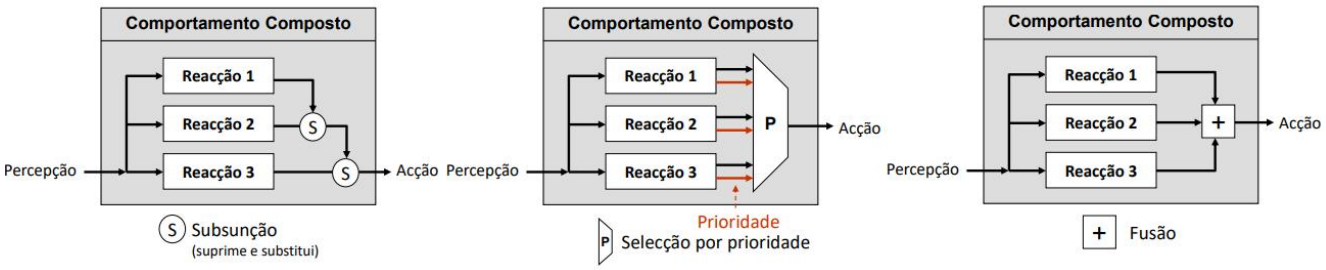


Figura 4 - Morgado, Luís. (2024). Esquema dos mecanismos de seleção de ação, Hierarquia, Prioridade e Combinação

Tal como foi referido anteriormente, estes agentes como tem um comportamento reativo, onde segundo uma perceção eles executam uma ação, isto pode levar a alguns problemas, como um comportamento cíclico que os leva a não conseguir progredir. Para evitar isso é necessário adicionar memória aos agentes.

Um agente reativo com memória, consiste num agente cujos comportamento envolvem uma máquina de estados interna, permitindo ao agente criar uma resposta ao ambiente não só através da sua perceção atual, mas também através de um histórico das perceções passadas.

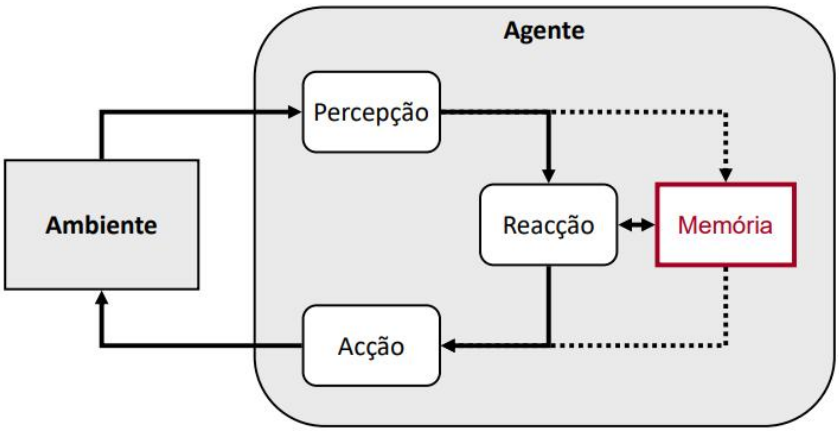


Figura 5 - Morgado, Luís. (2024). Arquitetura de um agente reativo com memória

Arquiteturas de agentes reativos com memória são capazes de produzir todo o tipo de comportamento, são capazes de evoluir com o tempo (o que pode levar a comportamentos mais complexos), estas arquiteturas também permitem os agentes tratar de comportamentos cíclicos, através da exploração de ações que não foram realizadas no passado. As desvantagens desta abordagem no agente, é que estes necessitam de memória e necessitam de a processar, o que aumenta a complexidade do sistema (de um modo espacial e computacional).

2.4 - Raciocínio Automático

Raciocínio automático é utilizado na resolução automática de problemas (área da inteligência artificial que através de algoritmos resolve problemas de uma forma automática). Este conceito refere-se à capacidade de sistemas computacionais realizar inferências (o método de manipular representações do conhecimento para assim obter conclusões) e resolver problemas automaticamente.

Este processo envolve dois tipos de atividades:

- **Exploração de opções** - processo de examinar as diferentes ações para a partir de uma situação inicial conseguir-se atingir um objetivo (e antecipar possíveis acontecimentos, raciocínio prospetivo). Para isto é necessário existir representações internas do domínio do problema, para assim ser possível simular internamente o domínio do problema. Domínio de um problema refere-se ao conhecimento que um agente possui sobre o ambiente em que ele opera, para assim conseguir interagir de uma forma eficaz com ele, para assim atingir os seus objetivos.
- **Avaliação de opções** - processo de determinar a qualidade das opções exploradas para resolver um problema, os critérios desta avaliação podem ser: o custo ou o valor. O custo

consiste nos recursos necessários para um agente ir de uma situação para outra, ou seja consiste no custo da realização de uma ação para transitar o estado do agente. O valor consiste no ganho ou perda associado a uma ação, pode ser medido segundo a utilidade ou beneficio que a ação teve na resolução de um problema.

Como já foi referido anteriormente, para resolver um problema segundo o raciocínio automático é necessário existir representações do domínio do problema, estas representações são internas ao sistema o que obriga "transformar" um problema em estruturas simbólicas que o permitem representar. Estas estruturas serão usadas para explorar e avaliar as opções possíveis para assim obter-se um solução ao problema. A existência destas estruturas leva a criação de processos de codificação e descodificação, para assim ser possível através das perceções de um agente, criar estas estruturas simbólicas para assim o agente conseguir executar a melhor solução para atingir o seu objetivo (solução do problema).

Modelo do problema consiste na representação do problema através de estruturas simbólicas que são mantidas na memória do sistema para assim encontrar-se um solução para o problema (através das informações destas estruturas). Este modelo serve de suporte ao raciocínio automático, este modelo é representado por estados, cada estado representa uma configuração possível de um problema, e por operadores, que representam ações que podem ser executadas por um agente que alteram o estado do problema (a aplicação de um operador sobre um estado permite a sua transição para outro estado). Através dos conceitos, estado e operadores, surge o conceito de espaço de estados, este consiste no conjunto de estados possíveis de um problema e as transições destes que permitem um estado transitar para outro.

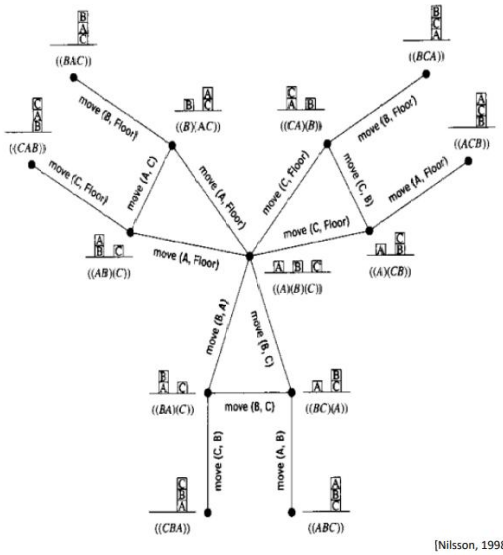


Figura 6 - Morgado, Luís. (2024). Exemplo de um espaço de estados representado sob a forma de um grafo

No raciocínio automático, os problemas são um dos conceitos principais (tal como os estados e operadores), sendo que estes são compostos por: um estado inicial (exemplo: ponto de partida de um agente), operadores e objetivos (estado que representa o objetivo do agente).

2.5 - Procura em Espaço de Estados

Procura em espaço de estados (PEE) é um método utilizado pelo raciocínio automático para a resolução de problemas, designado problemas de planeamento. Este método tem como objetivo obter uma sequência de estados e operadores que permitem ir de um situação inicial para uma final (ir do estado inicial até ao estado objetivo do problema), esta sequência corresponde á solução do problema.

O mecanismo de procura consiste na exploração do espaço de estados de uma forma sucessiva, partindo do estado inicial ("raiz") que será expandido através da aplicação de operadores, o que gera estados sucessores. Para cada estado sucessor verifica-se se este corresponde ao estado objetivo, se corresponder, a procura termina e retornar-se a sequência de estados e operadores que representam o "caminho" entre o estado inicial e o objetivo. Se o estado sucessor não corresponder ao objetivo, repete-se o processo de aplicação de operadores expandindo assim os estados. Caso não existem estados sucessores, o processo de procura termina, pois não foi encontrada uma solução (isto pode ocorrer quando todo o espaço de estados já foi explorado e não se encontrou uma solução ao problema).

Na PEE, existe uma estrutura designada por árvore de procura, esta tem como objetivo guardar a informação gerada em cada passo do mecanismo de procura. Esta estrutura baseia-se em nós que mantém a informação sobre o estado, o operador que o gerou e o estado antecessor (também pode guardar informações em relação a profundidade do nó na árvore, e o custo acumulado deste a raiz, estado inicial, até ao estado do nó). Dentro da árvore de procura, os nós que já foram explorados (expandidos) são designados por nós fechados. Os nós que ainda não foram explorados são designados por nós abertos e encontram-se na fronteira de exploração, é nesta fronteira que se determina a estratégia de procura/exploração dos nós, ou seja define-se qual dos nós vai-se expandir primeiro.

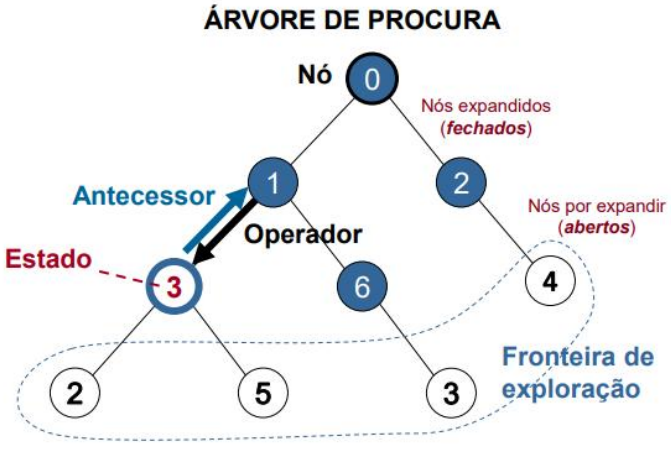


Figura 7 - Morgado, Luís. (2024). Exemplo de uma árvore de procura

Dependendo do método de procura utilizado, este pode ter critérios de ordenação diferentes na estratégia de controlo da procura. Como por exemplo, pode-se explorar primeiro os nós mais

recentes na fronteira de exploração recorrendo assim a uma fronteira do tipo LIFO (*Last-In, First-Out*), que tal como o nome indica os elementos mais recentes nesta fronteira serão os primeiros a sair dela. Este tipo de estratégia é utilizado em procuras em profundidade onde se explora primeiro os nós mais recentes, pois neste método os nós mais recentes são colocados no início da fronteira e remove-se o primeiro elemento da fronteira para este ser explorado (caso não seja o objetivo do problema).

Outro exemplo de estratégia de controlo de procura é explorar primeiro os nós mais antigos na fronteira de exploração recorrendo a uma fronteira do tipo FIFO (*First-In, First-Out*), nesta fronteira os nós mais recentes são inseridos no fim da fronteira e remove-se o primeiro elemento da fronteira para este ser testado (o primeiro elemento será o mais antigo) e caso este não seja o estado objetivo ele vai ser expandido. Este tipo de estratégia corresponde á estratégia utilizada em procuras em largura.

Existem vários métodos de procura, cada um com a sua estratégia de controlo de procura, na criação de um projeto que necessite estes mecanismos deve-se escolher o mecanismo mais adequado ao projeto em questão. Existem vários aspetos que podem ser utilizados para classificar estes métodos, como:

- **Completo** - um método de procura é considerado completo, se garantir que caso exista uma solução a um problema, esta será encontrada.
- **Óptimo** - métodos óptimos comprometem-se a devolver a melhor solução a um problema, caso esta exista (exemplo: garante retornar a solução que faz o agente dar o mínimo de passos possíveis até ao seu objetivo).
- **Complexidade** - os métodos podem ser classificados segundo a sua complexidade temporal, o tempo que demoram a encontrar um solução, e/ou a sua complexidade espacial, que consiste na memória necessária para obter uma solução.

Os métodos de procura também podem ser divididos em dois tipos:

- **Procura Não Informada** - estas procuras tem estratégias de exploração do espaço de estados, que baseiam-se numa procura não-guiada, ou seja guiam-se apenas pela informação disponível sobre os estados e operadores do problema. Explorando o espaço de estados de uma forma exaustiva, expandindo nós sucessores sem tirar partido do conhecimento do domínio do problema.
- **Procura Informada** - estas procuras tem estratégias de exploração do espaço de estados, que baseiam-se numa procura guiada, ou seja tiram partido do conhecimento do domínio do problema para assim orientar a sua exploração do espaço de estados em direção a uma solução. O conhecimento que estes mecanismos tem em relação ao problema é obtido segundo uma função designada por heurística que estima o custo de um nó em relação ao objetivo do problema.

2.5.1 - Procura em Profundidade

O mecanismo de procura em profundidade é um método de procura não informado, que consiste na exploração dos nós mais recentes primeiro, tal como foi referido anteriormente segue uma fronteira do tipo LIFO, colocando os nós mais recentes abertos no início da fronteira, sendo que estes vão ser explorados primeiro. Isto indica que a procura em profundidade faz uma pesquisa desde a raiz, estado inicial do problema, até um nó de um ramo da árvore não ter mais sucessores, caso não se encontre nenhum objetivo nesse ramo, volta-se a um nó antecessor capaz de expandir a árvore para assim continuar a exploração.

Em suma, este método de procura em profundidade é um algoritmo não completo, pois pode ficar preso em ciclos num ramo, também não é óptimo pois não garante a melhor solução, pois neste mecanismo a procura termina ao encontrar um solução, sendo que esta pode não ser a mais eficiente em termos de custo ou a mais curta.

Existe duas variantes deste mecanismo:

- **Procura em Profundidade Iterativa** - método que combina as vantagens da procura em profundidade com a garantia de encontrar uma solução óptima. Esta procura funciona de forma incremental, aumentando de uma forma progressiva o limite de profundidade a cada iteração até uma solução seja encontrada (na primeira iteração tem valor 0).
- **Procura em Profundidade Limitada** - método de procura que limita a profundidade máxima que pode ser explorada a partir do estado inicial. Esta limitação permite evitar ciclos infinitos e controla a quantidade de memória utilizada por este mecanismo.

2.5.2 - Procura em Grafos

O método de procura em grafos, evita a repetição da exploração de nós, para assim evitar desperdícios de recursos (como tempo e memória). Quando um nó na fronteira de exploração é expandido, verifica-se se o nó criado existe na fronteira de exploração ou nos nós fechados (aqueles que já foram explorados), se não existir em nenhum ele é inserido na fronteira de exploração. Caso exista um nó "igual" a ele nos nós abertos ou nos nós explorados, verifica-se qual deles tem o menor custo, se for o nó gerado ele é colocado na fronteira de exploração e o outro nó já existente será removido da árvore de procura. Se o nó já existente o que tiver menor custo, o nó gerado é descartado.

2.5.2.1 - Procura em Largura

O mecanismo de procura em Largura é um método de procura não informado, que consiste na exploração dos nós mais antigos primeiro, tal como referido anteriormente segue uma fronteira do tipo FIFO, colocando os nós abertos mais recentes no fim da fronteira de exploração, sendo que os nós mais antigos vão ser explorados primeiro. Isto indica que a procura em largura explora

primeiro todos os nós de uma certa profundidade para depois explorar nós mais profundos na árvore de procura.

Em suma, este método de procura em largura é um algoritmo completo, pois garante através da sua exploração sistemática que caso exista uma solução esta vai ser encontrada. Este mecanismo também é considerado óptimo pois encontra uma solução óptima em termos de número de passos, pois obtém a solução com menos profundidade. No entanto devido a sua procura exaustiva este método pode ter um grande custo computacional devido á sua complexidade espacial exponencial.

2.5.2.2 - Procura Melhor-Primeiro

O método de procura melhor-primeiro utiliza uma função para avaliação dos nós gerados/expandidos segundo o custo da solução através do nó, essa função é representada por **f(n)**, sendo que quanto menor o valor de **f(n)** mais promissor é o nó (n). Este método possui uma fronteira de exploração ordenada por ordem crescente segundo uma prioridade. Esta função pode se basear no custo dos nós explorados ou basear-se em estimativas de custo (heurística).

Heurística, representada pela função **h(n)** que consiste no valor estimado do custo do percurso desde o nó, n, até ao objetivo do problema. Esta função necessita apenas de conhecer o estado do nó e o estado objetivo do problema, para assim determinar um valor estimado do custo até ao objetivo sem que o percurso até este estado final tenha sido explorado, para isto ser possível é necessário existir conhecimento acerca do domínio do problema para assim orientar a procura até ao objetivo final.

2.5.2.3 - Procura de Custo Uniforme

O mecanismo de procura de custo uniforme é um método de procura não informado, e é uma variante da procura melhor-primeiro, neste método a função **f(n)** baseia-se apenas no valor do custo acumulado de cada nó (custo acumulado deste a raiz até ao nó), representado por **g(n)**. A fronteira de exploração nesta procura é ordenada segundo o custo acumulado de cada nó, sendo que o nó aberto com custo mais baixo será aquele que vai ser expandido primeiro, pois no momento este nó é a melhor opção em termos de custo. Este método como não utiliza a função heurística ele não tira partido do conhecimento do domínio do problema, sendo **f(n)** apenas afetado pelo custo até ao nó n, ou seja este método baseia-se exclusivamente no custo dos caminhos explorados.

Em suma, este método é um algoritmo completo, pois garante através da sua exploração que caso exista uma solução esta vai ser encontrada. Este mecanismo também é considerado óptimo pois encontra uma solução óptima em termos de custo, pois irá retornar a solução com menor custo possível.

2.5.2.4 - Procura Sôfrega

O método de procura sôfrega é um método de procura informado e uma variante da procura melhor-primeiro. Neste método a função **f(n)** baseia-se apenas na função **h(n)**, ou seja, no valor estimado do custo do nó até ao estado objetivo, ao contrário da variante anterior da procura melhor-primeiro, a procura sôfrega não tem em conta, na sua função de avaliação, o custo acumulado do nó, **g(n)**, preocupando-se apenas com o custo estimado para atingir o objetivo, sendo que isto pode levar a soluções sub-óptimas, pois esta procura pode não considerar caminhos que no inicio parecem menos promissores, mas que poderiam levar a uma solução com menor custo.

2.5.2.5 - Procura A*

O método de procura A* é um método de procura informado e uma variante da procura melhor-primeiro. Neste método a função **f(n)** baseia-se na soma do custo acumulado até ao nó, **g(n)**, e o custo estimado do nó até ao estado objetivo, **h(n)**. Esta variante como combina **g(n)** e **h(n)** na sua função de avaliação permite a exploração de uma forma mais eficiente do espaço de estados, obtendo uma solução óptima ao problema, para obter esta solução óptima também é necessário garantir a existência de uma heurística admissível, ou seja, o custo estimado de um nó até ao objetivo nunca pode ser superior ao custo real da solução. Este algoritmo também se considera completo, pois caso exista solução, ele a encontrará.

2.6 - Agentes Deliberativos

Tal como referido anteriormente no tópico 2.1 - Inteligência Artificial, numa arquitetura de um agente deliberativo o seu comportamento baseasse em processos de planeamento suportado por representações internas do ambiente (modelo do mundo), através das representações do domínio do problema, através de simulações internas explora-se as várias opções, para assim atingir-se os objetivos do agente (utilizando processos de deliberação para decidir quais objetivos realizar e como realizá-los). Estes agentes ao contrário dos reativos que apenas reagem a estímulos do ambiente com ou sem memória de ações passadas, ou seja agentes reativos só conseguem considerar o presente e caso tenham memória também o passado, os deliberativos também conseguem considerar o futuro, através da simulação de futuros estados, isto permite ao agente optimizar o seu comportamento no presente para assim atingir os seus objetivos.

Para os agentes deliberativos conseguirem considerar o passado, presente e futuro é necessário estes terem memória (componente fundamental na criação do comportamento deliberativo) para assim recordarem o passado, percecionar o ambiente e tomar ações no presente e simular futuras situações e preparar ações para estas. A memória também é necessária para estes agentes conseguirem ter um representação simbólica do ambiente.

Estes agentes tomam decisões segundo uma representação interna do ambiente, o modelo do mundo, e segundo os seus objetivos, sendo que esta representação do mundo pode ser atualizada segundo perceções do agente. Estes objetivos e o plano como estes vão ser concretizados são definidos segundo um raciocínio orientado para ações, também designado por raciocínio prático. Este raciocínio é composto por duas componentes:

- **Raciocínio sobre fins** - designado por deliberação, que segundo a perceção do agente tem de analisar os elementos que se encontram no ambiente/mundo e escolher aqueles que representam os objetivos a atingir.
- **Raciocínio sobre meios** - designado por planeamento, que segundo os vários objetivos tem de planear um plano de ações para assim conseguir atingir o resultado esperado.

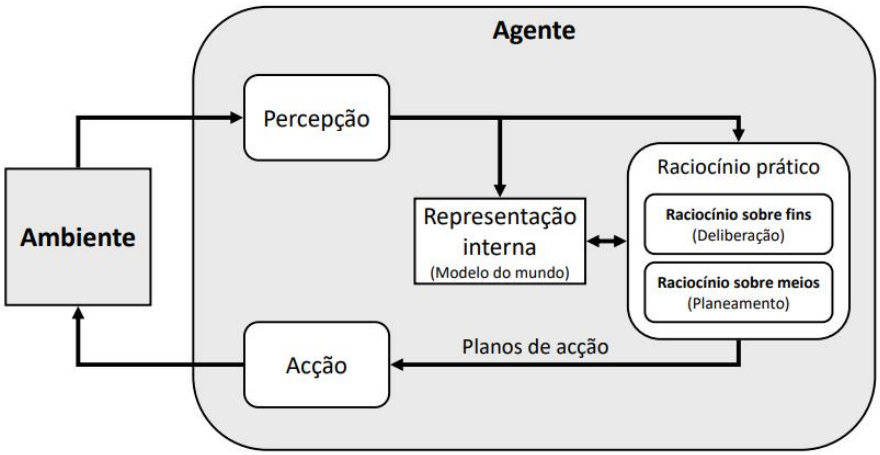


Figura 8 - Morgado, Luís. (2024). Arquitetura de um agente deliberativo

O raciocínio prático suporta o processo de tomada de decisão que indica segundo a perceção atual do ambiente e o estado do modelo interno do mundo, qual a ação que o agente deve executar para assim estar mais próximo dos seus objetivos. Este processo ocorre de um forma cíclica seguindo os seguintes passos: o agente perceciona o ambiente e segundo esta perceção atualiza o modelo do mundo, reconsidera-se as opções, ou seja devido as limitações dos recursos computacionais do agente deve-se confirmar durante o raciocínio se houve ou não alterações no ambiente que ainda não foram consideradas, se existir, através do raciocínio prático o agente delibera o que fazer, criando os objetivos a realizar pelo agente, de seguida planeasse como atingir esses objetivos, realiza-se um plano de ações, conjunto de ações que o agente tem de executar para atingir os objetivos. Por fim o agente executa o plano. Caso não haja nenhuma reconsideração a fazer e já exista um plano de ação, este é executado sem ser necessário o agente deliberar e voltar a planear o plano de ações.

O processo deliberativo que cria a sequência de ações para realizar os objetivos do agente (planos) designa-se por planeamento automático. Este planeamento pode ter como base, por exemplo, procuras em espaço de estados (tópico: 2.5 - Procura em Espaço de Estados). Este processo

necessita da definição dos objetivos a atingir e de um modelo de planeamento que é definido por:

- **Estado** - estado inicial do problema, que define uma situação ou um configuração do ambiente.
- **Estados** - conjunto de estados possíveis (por exemplo: as posições válidas de um agente num ambiente).
- **Operadores** - conjunto de operadores que permitem a transição de um estado para outro aplicando custos a cada transição.

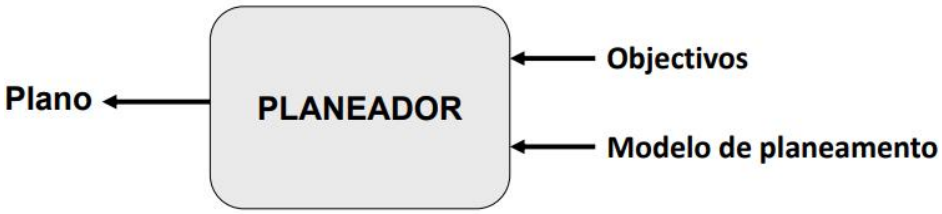


Figura 9 - Morgado, Luís. (2024). Planeamento automático PEE

2.7 - Processos de Decisão Sequencial

Um processo de decisão sequencial, consiste num processo onde a transição entre estados ocorre devido a ações, sendo que a aplicação destas ações podem criar resultados imprevisíveis (ações não deterministas). Isto significa que nestes processos a transformação do estado ocorre devido a ações com probabilidades (a probabilidade da ação "x" levar ao estado "y").

O Processo de Decisão de Markov (PDM), é um processo de decisão sequencial que utiliza a propriedade de Markov. Esta propriedade diz que o futuro é independente do passado, ou seja para prever os estados sucessores do estado atual de um agente apenas necessitamos do estado atual e não dos seus valores passados.

Com um modelo PDM é possível representar o mundo através dos seguintes componentes:

- **S** - conjunto de estados do mundo, ou seja conjunto de componentes que definem o mundo.
- **A(s)** - Conjunto das ações possíveis que podem ser tomadas num certo estado, s. Ou seja as ações que um agente pode tomar num certo estado.
- **T(s, a, s')** - Função de transição de estado, esta função define a probabilidade de transição de um estado s para um estado s', através de uma ação a.
- **R(s, a, s')** - Função de recompensa, esta função determina a recompensa ao transitar de um estado s para um estado s' através de uma ação a. A recompensa representa o ganho ou perda num determinado estado, s.
- γ - Taxa de desconto em relação ao tempo, determina a importância das recompensas futuras em comparação com as recompensas imediatas.

Um dos componentes fundamentais dos PDM, é a utilidade. A utilidade consiste na soma das recompensas segundo o estado atual do agente e as recompensas futuras esperadas. Estas recompensas podem ser aditivas, ou seja, apenas se soma o seu valor, não aplicando nenhum desconto, ou podem ser descontadas no tempo, γ.

Neste processos de decisão sequenciais, a forma de representar o comportamento do agente é através da política comportamental. Esta política define para cada estado qual a ação deve ser realizada. A política pode ser não determinista, se em cada estado existir uma probabilidade de execução de uma ação, ou pode ser determinista (não possui essa probabilidade), garantindo assim uma ação por estado.

Num Processo de Decisão de Markov, o princípio da solução óptima depende apenas do estado atual, o caminho de estados até este não é relevante para a decisão da ação óptima futura.

O planeamento automático com base em processos de decisão de Markov, define para cada estado possível a ação a realizar que maximiza a utilidade, através da política de ação. Este processo necessita da definição dos objetivos a atingir e de um modelo de planeamento que é definido por:

- **Estados** - conjunto de estados possíveis.
- **Operadores** - conjunto de operadores que permitem a transição de um estado para outro.

O planeador PDM também necessita da definição do modelo PDM interno para assim conseguir calcular a função de utilidade e da política. Este modelo é definido por:

- **S** - conjunto de estados possíveis.
- **A(s)** - conjunto de operadores.
- **T(s, a, s')** - Modelo da dinâmica.
- **R(s, a, s')** - Modelo de recompensa.

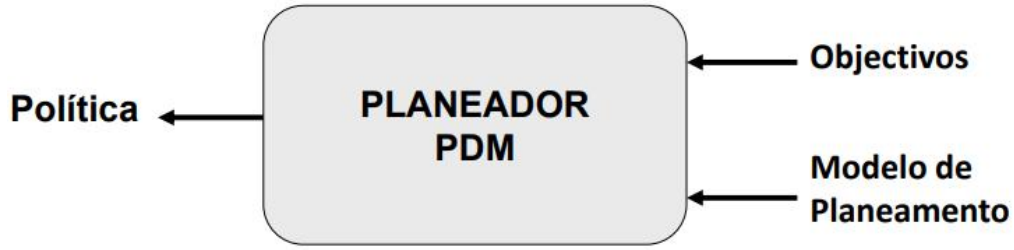


Figura 10 - Morgado, Luís. (2024). Planeamento automático PDM

3. Projecto - Parte 1

3.1 - Modelo do problema

A primeira parte deste projeto, teve com o objetivo a introdução de conceitos que envolvem a engenharia informática e conceitos que envolvem inteligência artificial, onde se implementou, em Java, um jogo com um agente reativo que interage com o jogador através da consola. Este jogo consiste num ambiente, onde o agente tem como objetivo registar animais ao fotografá-los.

3.2 - Desenvolvimento e implementação do sistema

Para o desenvolvimento do sistema que implementa este jogo, seguiu-se o processo de desenvolvimento de software, onde inicialmente se cria um modelo conceptual do sistema (Figura 6), para depois passarmos para a sua arquitetura (arquitetura de apoio ao projeto fornecida pelos docentes da cadeira).

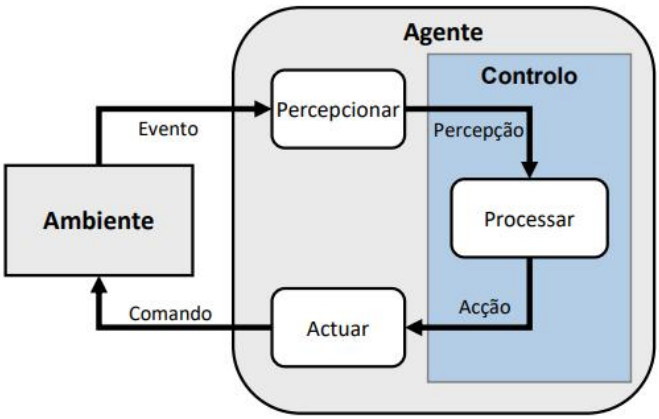


Figura 11 - Morgado, Luís. (2024). Modelo Conceptual da relação entre o agente e o ambiente

Depois de compreender-se o modelo e sua arquitetura, passa-se a implementação do código estrutural e comportamental do sistema (desenvolvidos dentro do package iasa_jogo). Onde inicialmente implementou-se classes que expressam conceitos que serão necessários para o nosso sistema, sem os especificar para o nosso problema (desenvolvidos nos package agente e ambiente), para assim criar-se código reutilizável e coeso simplificando a compreensão do mesmo e garantir a sua expansibilidade. Para isto utilizou-se interfaces para assim promover a organização do código de uma forma coesa, através do encapsulamento, elas também permitem uma melhor manutenção do sistema e estruturação do código, pois classes que implementam interfaces são obrigadas a implementar os métodos desta (promove um acoplamento fraco pois classes não tem de herdar diretamente uma das outras). As interfaces desenvolvidas são (3 primeiras interfaces são do módulo ambiente e a última do agente):

- `Ambiente` - representa um ambiente, em que um agente pode existir, este ambiente tem a capacidade de evoluir, permite um agente percecionar o ambiente e permite o jogador ver o que está a acontecer no ambiente.
- `Evento` - representa os possíveis eventos que podem ocorrer num ambiente, permite que o jogador ver a perceção do agente sobre o ambiente.
- `Comando` - representa as possíveis ações que um agente pode executar, segundo um estímulo do ambiente.
- `Controlo` - representa o controlo de um agente ou seja, as classes que implementa esta interface tratam do processamento da informação proveniente do ambiente (perceção do agente) e decide qual a ação adequada a executar.

No módulo do agente também foram desenvolvidas 3 classes:

- `Agente` - representa um agente reativo, que conhece o ambiente e tem um controlo, estes atributos privados da classe permitem o agente conhecer e interagir com o ambiente e através do controlo o agente consegue ter um comportamento inteligente executando ações segundo perceções do ambiente que o rodeia.
- `Percepcao` - representa uma perceção do agente, que conhece um evento do ambiente (um estado do ambiente que pode ser percecionado pelo agente).
- Accao - representa uma ação que o agente pode executar, que conhece um comando do ambiente (uma ação que o agente pode executar no ambiente).

Após o desenvolvimento de classes e interfaces que abrangem o modelo normal de um agente reativo, implementou-se classes que são especializações destas para implementar o jogo, package jogo. Este módulo é composto por um package ambiente, um package personagem e uma classe `Jogo` que tem um método que simula o jogo através de inputs do jogador (é o *user* que altera o estado do ambiente, ou seja ele altera aquilo que o agente consegue percecionar no ambiente), onde uma personagem (especialização da classe `Agente`) através do seu ciclo perceção-processamento-ação, irá percecionar o evento que ocorre no ambiente e tomar uma ação segundo ele. É esta a classe que possui uma `main` onde se cria um objeto do tipo `AmbienteJogo` (especialização da classe `Ambiente`) e `Personagem`, sendo que depois corre-se o método referido anteriormente (este método só termina o jogo quando o jogador escolher o evento que o termina).

O Módulo ambiente, consiste em 3 classes que implementam as interfaces `Ambiente`, `Evento` e `Comando`, sendo estas:

- `ComandoJogo` - esta classe é do tipo `enum (JavaEnumeration)` e implementa a interface `Comando`, decidiu-se usar um enumerador para representar o conjunto de ações que um agente pode executar neste ambiente virtual, pois os comandos do jogo são um conjunto finito, limitado e definido pelo programador . Esta classe implementa o método `mostrar()` que consiste num `print` da ação que o agente vai executar.

- `EventoJogo` - implementa a interface `Evento`, e tal como a classe `ComandoJogo` esta classe é do tipo `enum`, pois tal como as ações que o agente pode executar no jogo são o número limitado e definidas pelo programador, os estados que o jogo pode ter também o são.
- `AmbienteJogo` - é uma especialização de um ambiente, representa um ambiente virtual, ou seja um jogo. Este possui um atributo privado, o evento ou estado atual do jogo (`EventoJogo evento`) e um dicionário que utiliza um `HashMap` (que liga strings que representa os inputs aceitáveis pelo jogador a eventos do jogo). A abordagem deste dicionário simplifica o código, permitindo uma diminuição da complexidade do mesmo, uma melhor compreensão deste permite expansibilidade, pois para adicionar mais eventos só é necessário acrescentá-los ao dicionário e definir o Evento no enumerador. Outra abordagem possível para associar os inputs do jogador aos eventos poderia ser através de um `switch-case`, mas isto obrigaria a um aumento da complexidade do código no método `gerarEvento()`, função que recebe o input do jogador e retorna o `EventoJogo` escolhido. Esta classe também possui os seguintes métodos: `observar()` que mostra ao jogador o evento atual do jogo se ele não for `null`; `executar()` mostra a ação que o agente vai executar na consola, `evoluir()` que atualiza o evento do jogo segundo o método `gerarEvento()`e o `getEventoJogo()` que retorna o evento atual do jogo.

Input do jogador	s	r	a	f	o	t
Evento do Jogo	SILENCIO	RUIDO	ANIMAL	FUGA	FOTOGRAFIA	TERMINAR

Tabela 1 - Chaves (input) e valores (evento do jogo) do dicionário

O Módulo personagem, consiste em duas classes `Personagem` (como já referido anteriormente é uma especialização da classe `Agente`) e a classe `ControloPersonagem` que implementa a interface `Controlo.`

- `Personagem` - esta classe represente um agente inteligente dentro de um ambiente virtual, considera-se este agente inteligente, pois ele processa a informação que perceciona do ambiente e atua sobre este. Como esta classe é uma especialização da classe `Agente`, utiliza-se no construtor o `super()` onde se passa o ambiente que recebe como parâmetro, do tipo `AmbienteJogo`, e também passa-se uma instância do `ControloPersonagem`.
- `ControloPersonagem` - esta classe trata do processamento da informação do ambiente (perceção deste), e decide qual a ação mais adequada para o agente executar, segundo o evento atual do ambiente. É nesta classe que a personagem se torna inteligente seguindo o ciclo perceção-processamento-ação. Para representar o comportamento da personagem decidiu-se utilizar o mecanismo computacional designado por máquina de estados, nesta

classe apenas define-se os vários estados existentes e as suas respetivas transições (no construtor). Esta classe possui três métodos: `processar()` que segundo uma perceção atualiza o estado da máquina de estados e mostra a ação que a personagem vai executar; `mostra()` método privado usado no `processar()` para mostrar na consola a ação que o agente vai fazer; `getEstado()` retorna o estado atual da máquina de estados, este método usa delegação, pois chama o método público do atributo máquina de estados (`maquinaEstados`), criando assim redundância no código elaborado.

Tal como foi referido anteriormente para a implementação do controlo da personagem optou-se por desenvolver uma máquina de estados, para isso criou-se o módulo maqest onde se desenvolveu as seguintes classes:

- `Transicao` - esta classe representa uma transição entre dois estados, o que permite a máquina evoluir de um estado para o outro. Esta classe tem 2 atributos, o estado sucessor para permitir a atualização do estado na classe `MaquinaEstados` e também tem uma ação que será executada na transição entre estados.
- `Estado` - esta classe representa um estado de uma máquina de estados. Esta classe tem 2 atributos, uma `String` que representa o nome do estado e um dicionário (atributo privado chamado transicoes) onde se guarda as transições entre este estado e outros. A chave do dicionário é o evento do ambiente que faz a alteração entre estados e o valor um objeto `Transicao`, objeto que guarda informação sobre o estado sucessor e a sua ação (representa a função de saída, **λ**). Esta classe possui 4 métodos: duas assinaturas do método `transicao()`, um representa a mudança de estados (segundo a função de transição de estado, **δ**, método com 2 parâmetros, o evento do ambiente e o estado sucessor) e o outro representa a função de saída, **λ**, método com 3 parâmetros, o evento do ambiente, o estado sucessor e a ação que o agente toma na transição; um método `getNome()` para fazer leitura ao nome do estado; método `processar()` que retorna uma transição segundo o seu evento.
- `MaquinaEstados` - esta classe representa uma máquina de estados. Estas são utilizadas para representar um sistema computacional. A dinâmica de um sistema é criada através dos vários estados que o sistema pode ter e a forma como ele evolui ao longo do tempo, nesta classe a atualização do estado é feita no método `processar()`, onde o estado é atualizado através do valor do estado atual e pelo valor da entrada (`Evento` que ocorreu no ambiente). Utilizando factorização e modularização colocou-se grande parte da complexidade do sistema nesta classe. A desvantagem desta implementação é que não aceita guardas nas transições de estados (condição que inibe ou permite transições). Esta classe possui o método referido anteriormente e um `get` para o seu atributo `Estado`, que representa o estado atual na máquina de estados.

No jogo que foi desenvolvido nesta primeira parte do trabalho, foi nos dada informações sobre o conjunto de símbolos de entrada (∑ = Eventos do jogo), de saída (Z = Comandos do jogo: Procurar, Aproximar, Observar, Fotografar) e os estados que caracterizam a personagem (Q = nomes dos estados da máquina de estados do `ControloPersonagem`, Procura, Inspecção, Observação e Registo). O comportamento do sistema, máquina de estados desenvolvida no `ControloPersonagem` pode ser representado nas seguintes tabelas (na esquerda encontra-se a tabela que representa a função de transição de estado e na direita a tabela da função de saída do sistema):

Estado	Evento	Novo estado
Procura	Animal	Observação
Procura	Ruido	Inspecção
Procura	Silencio	Procura
Inspecção	Animal	Observação
Inspecção	Ruido	Inspecção
Inspecção	Silencio	Procura
Observação	Fuga	Inspecção
Observação	Animal	Registo
Registo	Animal	Registo
Registo	Fuga	Procura
Registo	Fotografia	Procura

Estado	Evento	Acção
Procura	Animal	Aproximar
Procura	Ruido	Aproximar
Procura	Silencio	Procurar
Inspecção	Animal	Aproximar
Inspecção	Ruido	Procurar
Inspecção	Silencio	
Observação	Fuga	
Observação	Animal	Observar
Registo	Animal	Fotografar
Registo	Fuga	
Registo	Fotografia	

Figura 12 - Morgado, Luís. (2024). Tabelas que representam função de transformação do sistema (δ e λ)

3.3 - Teste da aplicação

Depois da realizar toda a implementação da arquitetura, testou-se a aplicação para comprovar o seu bom funcionamento, inicialmente o personagem está num estado de procura, seguindo os inputs do jogador, podemos ver que o agente realiza as ações esperadas por isso conclui-se o seu bom funcionamento.

```
 Evento?
r
Percepção: RUIDO
Estado: Inspecção
Acção: APROXIMAR

 Evento?
a
Percepção: ANIMAL
Estado: Observação
Acção: APROXIMAR

 Evento?
a
Percepção: ANIMAL
Estado: Registo
Acção: OBSERVAR
```

```
 Evento?
a
Percepção: ANIMAL
Estado: Registo
Acção: FOTOGRAFAR

 Evento?
f
Percepção: FUGA
Estado: Procura

 Evento?
t
Percepção: TERMINAR
Estado: Procura

Process finished with exit code 0
```

Figura 13 - Teste do jogo

4. Projecto - Parte 2

4.1 - Modelo do problema

A segunda parte deste projeto, teve com o objetivo a implementação de uma agente reativo, desta vez em Python. Onde este será autónomo (não necessita de *inputs* do jogador para reagir a estímulos e ativar as suas respostas a estes), sendo a sua meta explorar o ambiente recolhendo os alvos que encontra e evitar os obstáculos no seu caminho.

4.2 - Desenvolvimento e implementação do sistema

Este sistema apresenta uma maior complexidade, em relação ao primeiro trabalho, logo para seguir as boas práticas de desenvolvimento de software, iremos utilizar modularização para dividir o projeto em vários módulos, este projeto será desenvolvido no package iasa_agente, sendo que dentro deste vamos inicialmente trabalhar num *package* que envolve as abstrações dos conceitos teóricos que irão nos permitir atingir, através das suas especificações, o comportamento do agente desejado. Este package lib (bibliotecas) será composto pelos seguintes módulos:

- ECR - Esquemas Comportamentais reativos, é um *package* criado pelos alunos, que envolve classes que expressam conceitos necessários para o nosso sistema. Como os estímulos que um agente reativo obtém de um ambiente ao percecioná-lo, as respostas associadas a estes que levam a ações dos agentes. Tal como as associações destes temas, os comportamentos e mecanismos para a seleção de ações, no caso de um comportamento ter várias associações estímulo-resposta (reações).

- SAE - Simulador de Ambiente de Execução, é um *package* fornecido pelo docente da cadeira, que consiste na implementação em Python, das abstrações dos conceitos teóricos da parte 1 do projeto, em conjunto com a implementação do ambiente (classe `Simulador`) e agente (classe `Agente`) que serão usados nesta segunda parte do projeto.

Para o desenvolvimento dos conceitos referidos no *package* ECR, começou-se por desenvolver as classes abstratas (são abstratas e não interfaces pois em Python não existe um recurso que permite a implementação de interfaces) `Comportamento` e `Estimulo` e as classes `Resposta`, `Reaccao`, `ComportComp`, `Hierarquia` e `Prioridade`:

- `Comportamento` - classe abstrata que representa um módulo comportamental (um comportamento), este pode ser um conjunto de reações ou apenas uma, com o objetivo de atingir um certo resultado. Para isto um comportamento tem de conseguir relacionar os padrões de perceção com os de ação. Esta classe possui um método abstrato `activar()`

que tem como finalidade ativar uma ação segundo uma perceção do agente no ambiente.

- `Estimulo` - classe abstrata que representa um estímulo detetado por um agente. Um estímulo é um elemento de uma perceção que leva a ações por parte do agente. Esta classe possui um método abstrato `detectar()` que quando implementado, será capaz de detetar estímulos segundo perceções.

- `Resposta` - classe que representa uma resposta a um estímulo do ambiente, o agente responde a esses estímulos executando uma certa ação predefinida (`accao` atributo protegido da classe), ou seja, uma resposta produz uma ação que será executada por um agente. Esta classe tem um método `activar()` que ativa uma ação alterando a sua prioridade, caso a perceção exista. Quando maior for a intensidade de um estímulo, maior será a prioridade da ação, sendo assim, a execução da ação do agente depende da intensidade do estímulo.

- `Reaccao` - esta classe representa uma reação, ou seja uma relação entre o estímulo e uma resposta (associação estímulo-resposta), logo tem uma variável privada do tipo `Estimulo` e outra do tipo `Resposta`, esta classe também possui um método `activar()`, que segundo uma perceção do ambiente obtém a intensidade de um estímulo. Se esta intensidade for maior que zero, significa que foi detetado um estímulo, logo ativa-se uma resposta para o agente conseguir reagir a este e tomar uma ação.

- `ComportComp` - esta classe representa um comportamento composto, logo herda da classe `Comportamento`. Um comportamento composto é aquele que reúne várias reações (comportamentos simples) e também pode possuir outros comportamentos compostos, logo esta classe tem um atributo privado comportamentos que consiste numa lista do aglomerado de comportamentos. Esta classe possui dois métodos, `activar()` que segundo uma perceção ativa uma ação. O estímulo criado segundo a perceção do ambiente, leva a uma ação por parte do comportamento. Depois dentro de todos os comportamentos existentes no comportamento composto, escolhe-se a ação mais adequada para o agente a executar (utiliza o método `seleccionar_accao()` da classe); também tem um método abstrato `seleccionar_accao()` que tem como objetivo segundo uma lista de ações escolher a mais adequada (esta escolha depende do mecanismo desejado).

- `Hierarquia` - classe que implementa o mecanismo de seleção de uma ação por hierarquia. Neste mecanismo a prioridade de cada ação é predefinida (estática), a ação que resulta da reação de mais alto nível, é aquela que o agente executa. Para facilitar a implementação dos mecanismos de seleção do comportamento composto, esta classe e a classe

`Prioridade` são ambas especializações de Comportamento Composto (`ComportComp`). Esta classe implementa o método abstrato `selecionar_accao()`. Nesta implementação considera-se que o primeiro elemento da lista de ações (argumento do método) é a reação de mais alto nível, suprimindo assim as outras.

- `Prioridade` - esta classe implementa o mecanismo de seleção de uma ação segundo a sua prioridade. Neste mecanismo a prioridade de cada ação é dinâmica, ou seja através da perceção do ambiente, é implementada uma prioridade para cada reação, sendo que a ação com maior prioridade é executada pelo agente. Tal como a classe `Hierarquia`, esta implementa o método abstrato `selecionar_accao()`. Nesta implementação utiliza-se a função `max()` do Python com uma função lambda, para obter a ação dentro da lista de ações (argumento do método) com a maior prioridade.

Depois de implementar os conceitos teóricos sobre o agente reativo e os mecanismos que permitem a escolha de uma ação, criou-se (fora do package lib) o package agente, dentro deste package vão ser implementadas especializações dos conceitos abordados para assim criarmos um agente que consiga executar os objetivos deste projeto (criação de um agente que recolhe alvos, evita obstáculos e explora o ambiente). Dentro do package agente criou-se o package controlo_react, para assim implementarmos um controlo de um agente reativo, ou seja um controlo que permite o processamento das perceções segundo um módulo comportamental (comportamento geral do agente). Este comportamento do agente pode ser formado por diferentes sub-comportamentos.

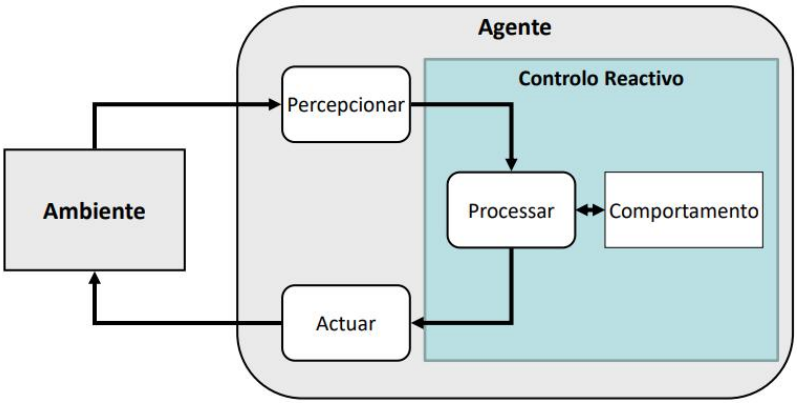


Figura 14 - Morgado, Luís. (2024). Esquema de um agente com um controlo reativo

Este conceito de controlo reativo foi implementado na classe `ControloReact` que consiste numa especialização da classe abstrata `Controlo` (feita pelo docente). Esta classe tem dois atributos, um atributo privado que representa o comportamento geral do agente segundo a sua perceção do ambiente e um atributo público booleano usado para permitir que seja visível a sua visão sensorial durante os testes da aplicação. Esta classe também tem um método `processar()` que consiste em processar o ambiente segundo a perceção do agente e obter a ação a executar por este.

Para se atingir os objetivos deste segunda parte é necessário modelá-los sob a forma de comportamentos, sendo que o agente tem o objetivo de recolher alvos, logo este será o nosso comportamento. Ele é composto por sub-comportamentos, sendo estes Aproximar Alvo, Evitar Obstáculos e Explorar, pois estes sub-comportamentos são necessários para que o agente seja capaz de recolher os seus alvos. A implementação destes sub-comportamentos foi feita dentro de um package reaccoes, dentro do controlo_react (dentro destes package, é criado um package para cada comportamento para assim organizar as classes criadas).

Inicialmente, implementa-se a especialização de uma Resposta (no seu package respetivo, resposta), que permita que o agente se movimente no ambiente, isto é um requisito obrigatório para este conseguir se deslocar no ambiente e recolher os seus alvos. `RespostaMover()` é uma especialização de uma `Resposta` que permite o movimento do agente segundo uma certa direção. Sendo que a `RespostaMoverAleat()` é uma especialização desta que permite que a direção do movimento do agente seja escolhida de uma forma aleatória.

O sub-comportamento Explorar, foi implementado no seu respetivo package. A classe `Explorar`, uma especialização de um `Comportamento`, representa um comportamento fixo que consiste no movimento do agente de uma forma aleatória. Um comportamento fixo é aquele que não tem estímulo, mas tem uma resposta fixa, ou seja o agente não precisa percecionar o ambiente para o explorar. Esta classe tem um método `activar()` que consiste na deslocação do agente numa direção aleatória, para isto é criado uma instância de uma `RespostaMoverAleat()` e retorna-se a sua ativação.

O sub-comportamento Aproximar Alvo, foi implementado no seu respetivo package. Para a implementação deste comportamento foram implementadas 3 classes, `AproximarAlvo`, `AproximarDir` e `EstimuloAlvo`:

- `EstimuloAlvo` - esta classe implementa a classe `Estimulo`, representando um estímulo que permite ao agente, ao percecionar o ambiente numa certa direção, detetar um alvo. Esta classe tem dois atributos privados a direção onde se podem encontrar alvos e um atributo que permite quantificar a intensidade de um estímulo, por omissão o valor de gama é 0.9. Esta classe implementa o método abstrato `detectar()`, ele segundo um perceção do ambiente permite detetar estímulos. Este método possibilita o agente detetar um alvo segundo uma direção (atributo privado da classe `__direccao`), caso ele encontre um alvo quanto maior for a sua distância, menor vai ser a intensidade do estímulo (utiliza-se o atributo gama, para se quantificar a intensidade do estímulo, sendo que o seu valor é inversamente proporcional à distância), se não encontra-se nenhum alvo a intensidade do estímulo é 0.
- `AproximarDir` - esta classe é uma especialização de uma `Reaccao`, ela representa uma reação que permite ao agente, caso encontre um elemento do tipo alvo possui uma resposta a este estímulo sendo esta a sua deslocação na sua direção. No construtor utiliza-

se o `super()`, para associar uma instância de um `EstimuloAlvo` e uma de `RespostaMover` numa certa direção.

- `AproximarAlvo` - esta classe herda da classe `Prioridade` e representa um comportamento composto por sub-comportamentos específicos para assim o agente conseguir aproximar-se dos seus *targets* através de uma das direções (NORTE, SUL, ESTE, OESTE). Esta classe instância no seu construtor um comportamento `AproximarDir` para cada direção que o agente consegue-se se movimentar (segundo o enumerador da classe `Direccao`), sendo que utiliza-se o `super()` para guardar essa lista de comportamentos, pois a prioridade, nesta implementação, é uma especialização de um comportamento composto. A seleção de uma ação deste comportamento é feita através do mecanismo de seleção, Prioridade. Onde o agente ao percecionar o ambiente, cada comportamento obtém um estímulo, aquele que tiver uma maior intensidade significa que o agente está mais próximo de um alvo nessa direção que esse comportamento corresponde, logo este vai ser o comportamento com a ação prioritária a ser executada.

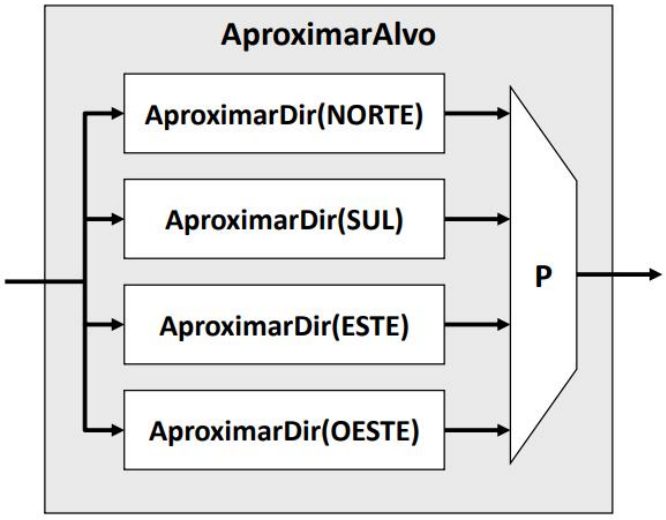


Figura 15 - Morgado, Luís. (2024). Comportamento composto AproximarAlvo

O sub-comportamento Evitar Obstáculos, foi implementado no seu respetivo package. Para a implementação deste comportamento foram implementadas 4 classes, `EvitarObst`, `EvitarDir`, `EstimuloObst` e `RespostaEvitar`:

- `EstimuloObst` - esta classe implementa a classe `Estimulo`, representando um estímulo que permite ao agente, ao percecionar o ambiente numa certa direção, detetar um obstáculo. Esta classe tem dois atributos privados a direção onde se podem encontrar alvos e a intensidade, intensidade do estímulo percecionado pelo agente (por omissão tem valor de 1). Esta classe implementa o método `detectar()`, este permite detetar um estímulo segundo uma perceção (argumento do método). Esta função possibilita o agente detetar um obstáculo segundo uma direção, caso ele encontre o obstáculo retorna-se a intensidade deste, se não encontrar retorna 0.

- `RespostaEvitar` - esta classe herda da classe `RespostaMover` (especialização de uma resposta a um estímulo que permite o movimento do agente numa direção). Esta classe permite ao agente ao percecionar o ambiente deslocar-se com o objetivo de evitar obstáculos. Utiliza-se o construtor da classe `RespostaMover`, para se criar uma Accao segundo uma direção inicial (`dir_inicial` por omissão deste argumento a direção inicial tem o valor de `Direçcao.ESTE`), também se guarda num atributo privado `__direccoes` a lista de direções possiveis para o movimento do agente. Esta classe tem 2 métodos, `__direccao_livre()` método privado que é utilizado no outro método `activar()`, que segundo uma perceção do ambiente, obtém uma lista das direções livres (sem obstáculos) e retorna aleatoriamente uma das direções livres. O método `activar()` segundo uma perceção do ambiente deteta um obstáculo na direção da sua ação atual, caso isso aconteça é atualizada a ação que o agente deve executar. Atualiza-se a sua direção segundo uma direção livre para assim o agente conseguir evitar os obstáculos. No `return` deste método usa-se o `super()` para assim chamar o método `activar()` da classe `Resposta` para assim atualizar a prioridade da `accao` segundo a intensidade do estimulo.

- `EvitarDir` - esta classe é uma especialização de uma `Reaccao`, ela representa uma reação que permite ao agente, caso encontre um elemento do tipo obstáculo possui uma resposta a este estímulo sendo esta a sua deslocação numa direção para assim evitar o elemento. No construtor utiliza-se o `super()`, para associar uma instância de um `EstimuloObst` e uma resposta a este estímulo.
- `EvitarObst` - Esta classe herda da classe `Hierarquia`, ela instância comportamentos com memória, `EvitarDir`, para cada direção que o agente pode se movimentar, ou seja esta classe representa um comportamento composto constituído por sub-comportamentos específicos para assim o agente conseguir evitar obstáculos através de uma das direções (NORTE, SUL, ESTE, OESTE, direções do enumerador `Direccao`). No método construtor, utilizando o método `super()` guarda-se a lista de comportamentos (instâncias de `EvitarDir` para cada uma das direções existentes na classe `Direccao` - NORTE, SUL, ESTE, OESTE) que fazem parte do comportamento composto `EvitarObst`. Neste construtor também se instância uma `RespostaEvitar` (atributo privado `__resposta`), uma resposta a um estímulo que permite o agente evitar obstáculos, sendo que este atributo é passado a cada comportamento `EvitarDir()`, para assim o agente conseguir evitar obstáculos em todas as direções.

Como todos os sub-comportamentos foram implementados é possível implementar o comportamento compostos Recolher, criou-se uma classe `Recolher` que herda da classe `Hierarquia`, onde no seu construtor instância-se os três sub-comportamentos, na seguinte ordem `AproximarAlvo()`, `EvitarObst()` e `Explorar()`.

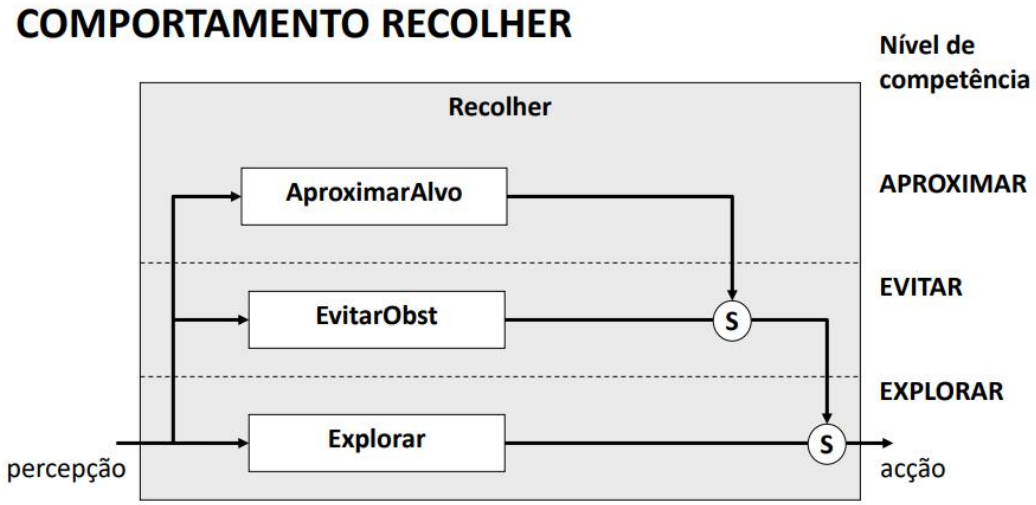


Figura 16 - Morgado, Luís. (2024). Comportamento Recolher

4.3 - Teste da aplicação

Depois da realizar toda a implementação da arquitetura, testou-se a aplicação para comprovar o seu bom funcionamento, inicialmente o agente tem no seu campo de visão 2 alvos, onde ele se dirige para o mais perto, que se encontra na direção Sul. No fim da simulação, o agente já recolheu todos os seus alvos portanto fica a vaguear pelo o mapa, num estado de exploração.

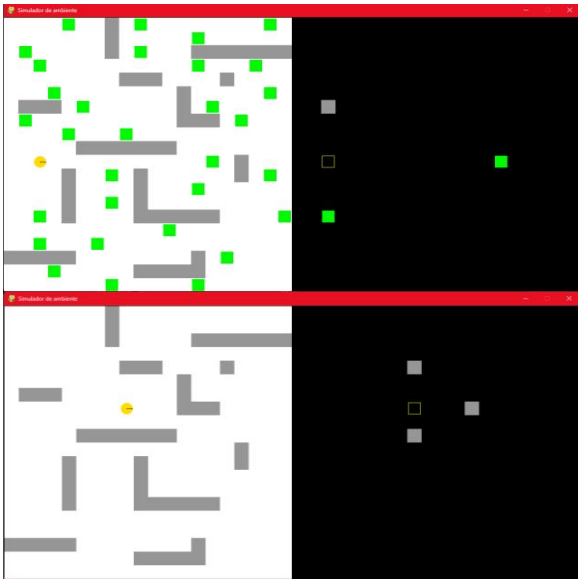


Figura 17 - Teste do Sistema

5. Projecto - Parte 3

5.1 - Modelo do problema

A terceira parte deste projeto, teve com o objetivo a implementação dos métodos de procura em espaço de estados, em Python. Sendo que no fim desta parte fez se testes em consola, para confirmar o bom funcionamento destes métodos.

5.2 - Desenvolvimento e implementação do sistema

Este sistema apresenta uma maior complexidade, em relação ao duas primeiras partes deste projeto, logo para seguir as boas práticas de desenvolvimento de software, iremos utilizar modularização para dividir o projeto em vários módulos, este projeto irá continuar a ser desenvolvido no package iasa_agente, sendo que dentro deste vamos inicialmente trabalhar num *package* que envolve as abstrações dos conceitos teóricos que irão nos permitir atingir, através das suas especificações, o comportamento do agente desejado. Este package lib (bibliotecas) é composto pelas pastas ECR (parte 2) e SAE (*package* fornecido pelo docente da cadeira, que representa a parte 1 do projeto) a estes módulos vamos adicionar os seguintes:

- MOD - package Modelo, é um *package* criado pelos alunos, que envolve classes que expressam conceitos necessários a parte 3 e 4 do projeto, pois este package é composto pelos elementos base da representação de um problema (modelo de um problema). Como os estados, configurações de um problema, os operadores que representam ações que alteram o estado do agente e os problemas, conceito fundamental do raciocínio automático composto por um estado inicial, operadores e um estado objetivo.

- PEE - package Procura em Espaço de Estados, é um *package* criado pelos alunos, que envolve classes que expressam conceitos necessários para a procura em espaço de estados, tal como a fronteira, fronteira de exploração de uma árvore de procura, uma classe que representa o mecanismo de procura. Dentro deste package também encontram-se outros package, um para a procura em profundidade e as suas variantes (limitada e iterativa), outro para a largura e outro para procura melhor-primeiro e suas variantes (custo uniforme, sôfrega e A*).

Para o desenvolvimento dos conceitos referidos no *package* MOD, começou-se por desenvolver as classes abstratas `Estado`, `Operador` e `Problema`:

- `Estado` - classe abstrata que representa um estado, ou seja uma configuração de um problema, esta classe possui um método abstrato `id_valor()` este método quando for implementado será capaz de identificar cada estado com um identificador único. Esta

classe também possui outros dois métodos `__hash__()` e `__eq__()`, o método `__hash__()` permite retornar um valor único para um objeto do tipo estado (no *return* usa o método abstrato). O método `__eq__()` é um método do Python que permite a comparação de dois objetos do tipo Estado através do seu identificador único, este método será útil para compararmos os estados, ou seja as configurações de um problema.

- `Operador` - classe abstrata que representa uma ação que permite a transformação/alteração de um estado para outro. Esta classe possui dois métodos abstratos `aplicar()` e `custo()`, o método aplicar tem o intuito de produzir uma transição do estado que recebe como argumento. Enquanto que o método custo tem como objetivo indicar o custo de transição entre um estado e o estado sucessor a este.
- `Problema` - classe abstrata que representa um problema, um dos conceitos principais de um Raciocínio automático, este conceito é constituído por: um estado inicial (estado inicial do agente, ponta de partida do raciocínio), uma lista de operadores, um conjunto de ações que permitem a alteração de um estado e o objetivo, estado que representa o objetivo do agente. Esta classe possui dois atributos privados o estado inicial do problema e a lista de operadores, ambos os valores são guardados no construtor. Esta classe possui um método abstrato `objectivo()` que quando implementado irá compara o estado que recebe com o parâmetro com o objetivo do problema, para assim o método de procurar saber se o objetivo do problema já foi ou não atingido. Esta classe também tem duas propriedades, uma para cada atributo privado, estado inicial e operadores.

Depois do desenvolvimento dos conceitos referidos no *package* MOD, começou-se por desenvolver as classes do *package* PEE e depois desenvolveu-se os restantes *packages* dentro deste:

- `No` - classe que representa um nó, ou seja um elemento de uma árvore de procura. No seu construtor, guarda-se em atributos privados o valor do estado representado pelo nó, o operador que levou ao estado (opcional), o nó antecessor (opcional) e o custo acumulado até ao nó (opcional). Caso exista um nó antecessor utiliza-se a profundidade desse nó na árvore de procura para se obter a do novo nó instanciado (é a profundidade do nó anterior mais um pois um nó aberto que é explorado, os seus nós sucessores vão ser sempre de um nível de profundidade mais alto), se não existir um nó antecessor, a profundidade do nó é zero. Esta classe possui 5 propriedades, uma para cada atributo privado mencionado no construtor. Esta classe também possui um método `__lt__()` que permite dois objetos da classe No serem comparados segundo o operador "<", esta comparação é feita segundo o custo acumulado de cada nó.
- `PassoSolucao` - classe de dados "`@dataclass`" que representa um classe que agrega dados, sendo estes dados uma instância de um Estado e uma de um Operador (operador que altera o estado para um estado sucessor).
- `Solucao` - classe que representa uma solução de um certo problema, esta solução

consiste numa sequência de estados e operadores que liga o estado inicial do problema com o estado que representa o objetivo. No construtor desta classe, guarda-se o valor do nó final e cria-se uma lista privada, que consiste na sequência de estados e operadores referidos anteriormente. Para isto faz se um ciclo `while` onde se testa se o nó tem antecessores, quando este não tiver antecessor significa que atingimos a raiz da árvore de procura então paramos a sequência, pois já obtemos a informação de todos os nós da árvore de procura deste o final até ao estado inicial. Esta classe possui duas propriedades, uma que retorna a dimensão do percurso do nó final, `dimensao()`, e outra que representa o custo da solução `custo()`. Esta classe também possui dois métodos __iter__ , que permite a solução ser iterável segundo os seus passos e __getitem__ método que permite aceder os passos da solução através de índices.

- `Fronteira` - classe abstrata que representa uma fronteira de exploração, ou seja um nível de nós que ainda não forma explorados dentro de uma árvore de procura. Uma árvore de procura baseia-se em nós , em cada nó têm-se informação sobre o estado, o operador que resultou esse estado e o seu antecessor. Sendo que a fronteira de exploração é uma parte da árvore que consiste em nós por expandir (abertos). Esta classe tem um construtor que chama o método `iniciar()` que inicializa um lista privada vazia. Esta classe também possui um método abstrato `inserir()` que quando implementado, segundo a estratégia de procura do método, este método irá inserir um nó no inicio ou no fim da lista. Esta classe também possui o método `remover()` que remove e retorna o primeiro elemento da lista de nós e possui uma propriedade que testa se o lista de nós é vazia ou não.
- `MecanismoProcura` - classe abstrata que representa um mecanismo de procura, ou seja um mecanismo que segundo um problema, ele vai percorrer um conjunto de estados possíveis para a resolução do problema (espaço de estados) até encontrar uma solução, caso esta exista. Esta classe tem um método construtor onde guarda a fronteira de exploração num atributo privado. Tem 3 métodos e um método abstrato, o método abstrato quando implementado irá adicionar um nó á fronteira de exploração. O outros 3 métodos são: `_iniciar_memoria()`, coloca a lista de nós da fronteira vazia, método `_expandir()` que recebe o problema e o nó a explorar, este método expande a árvore de procura através do nó que recebe como parâmetro, percorre-se todos os operadores que existem no problema, onde se aplica cada um dos operadores ao estado do nó, se o estado sucessor não for `None` calcula-se o custo do novo nó, para depois o instanciar e o adicionar a uma lista. O terceiro método é o `procurar()` , este método procura uma solução segundo um problema. Inicialmente inicializa-se uma lista de nós vazia, e instância-se um nó segundo o estado inicial do problema, este depois será guardado na fronteira (método abstrato que depende do mecanismo de procura). Percorre-se a fronteira de exploração até esta for vazia com o objetivo de encontrar uma solução para o problema, para isto

remove-se o nó da fronteira e testa-se, se o estado deste nó é o objetivo deste problema. Se o estado for o objetivo do problema instância-se uma Solução segundo o nó removido, se não, expande-se a árvore de procura e atualiza-se a fronteira de exploração (com os nós filhos) para assim repetir-se o ciclo da procura pelo estado que representa o objetivo do problema.

- ProcuraGrafo - classe abstrata que é uma especialização de um mecanismo de procura, sendo este o método de procura em grafos. O mecanismo de procura em grafos, evita a repetição da exploração de um nó através dos seguintes verificações:

  - Verifica se um nó não existe na fronteira de exploração (nós abertos), nem nos nós já expandidos (nós fechados). Neste caso, este nó gerado é adicionado á fronteira de exploração.

  - Verifica se um nó gerado pertence á fronteira de exploração, se sim verifica se este nó tem um menor custo que o nó existente na fronteira, se tiver o nó aberto é removido e insere-se o nó gerado na fronteira.

  - Verifica se um nó gerado pertence aos nós expandidos, se sim verifica se este nó tem um menor custo que o nó existente, se tiver o nó fechado é removido e insere-se o nó gerado na fronteira.

  Nesta implementação do mecanismo apenas se verifica se um nó já foi explorado pelo método, através da criação de um dicionário onde se guarda o estado do nó e o seu respetivo nó. Isto permite ignorar a exploração de nós que já existem no dicionário.

  Esta classe possui três métodos: `_iniciar_memoria()`, utiliza o super para colocar a lista de nós na fronteira vazia, também inicializa um dicionário que representa os nós já explorados. O método `_memorizar()`, adiciona nós á fronteira de exploração e ao dicionário de nós explorados, caso o estado deste nó não exista no dicionário dos nós explorados, o que indica que este ainda não foi explorado. O método `_manter()`, método que indica se um nó deve ser descartado ou guardado, retorna `true` se o nó não existe no dicionário e `false` se já existe.

Depois do desenvolvimento dos conceitos referidos anteriormente, começou-se por desenvolver as classes do *package* prof que se encontra dentro do *package* PEE:

- `FronteiraLIFO` - esta classe herda da classe abstrata Fronteira, sendo que representa uma fronteira de exploração onde os seus elementos são inseridos e removidos segundo a lógica LIFO (Last-In-First-Out). Ou seja, o último nó inserido na fronteira será o primeiro a ser removido desta. Esta classe possui um método, `inserir()` insere um nó no inicio da lista de nós da fronteira.
- `ProcuraProfundidade` - esta classe é uma especialização de um mecanismo de procura, sendo este o mecanismo de procura em profundidade (método de procura não informada). Este tipo de mecanismo consiste em explorar os nós mais recentes primeiro, ou seja

explora-se um ramo da árvore de procura até o nó explorado não ter mais sucessores, neste caso, volta-se para um nó antecessor (nó capaz de expandir a árvore) para continuar a exploração. Este mecanismo consiste nos seguintes passos:

1. Começa-se com o nó inicial e este é colocado na fronteira de exploração.

2. Exploração em profundidade: Até se encontrar um estado do nó que corresponde ao estado objetivo, repete-se os seguintes passos:

- Remove-se o nó mais recente da fronteira (primeiro elemento).

- Com a aplicação de operadores sobre o estado do nó removido, gera-se os seus nós sucessores.

- Adiciona-se esses nós sucessores à fronteira (os nós mais recentes são colocados no início da fronteira de exploração, pois segundo a estratégia de controlo estes nós, como são mais profundos, colocam-se no inicio da lista).

3. Verificação do objetivo: Durante a exploração, sempre que um nó é removido da fronteira, verifica-se se o estado de este corresponde ao estado objetivo. Se for, a procura termina.

Esta classe possui um construtor onde se instância uma `FronteiraLIFO()`. Esta classe também possui o método `_memorizar()` que adiciona um nó a fronteira de exploração.

- `ProcuraProfLim` - Esta classe é uma especialização do mecanismo de procura em profundidade, representando o mecanismo de procura em profundidade limitada.

Este método de procura restringe a profundidade máxima dos nós que serão explorados. Isso é útil para evitar a exploração excessiva em profundidades muito grandes, o que pode levar a um consumo excessivo de recursos. Esta classe possui um construtor onde se utiliza o `super()` pra criar uma fronteira que segue a lógica LIFO e guardamos numa variável privada o valor da profundidade máxima. Também temos um método o `_expandir()` que é responsável por expandir os nós na árvore de procura. Caso a profundidade do nó for menor que a profundidade máxima utiliza-se o método da classe abstrada mecanismo de procura, para expandir a árvore de procura segundo o nó recebido, se não retorna-se uma lista vazia. Esta classe também tem uma propriedade e um setter sobre o atributo profundidade máxima.

- `ProcuraProfIter` - esta classe é uma especialização do mecanismo de procura em profundidade limitada, representando o mecanismo de procura em profundidade iterativa.

Este mecanismos é uma método que combina os benefícios do mecanismo de procura em profundidade com a garantia de encontrar uma solução óptima. Este método expande gradualmente o limite de profundidade, explorando mais a fundo em cada iteração, até que a solução seja encontrada. Isso garante que a solução encontrada seja a mais profunda possível, garantindo que esta seja óptima. Esta classe possui um método `procurar()` que é responsável por realizar a busca em profundidade iterativa. Este método realiza uma busca em profundidade iterativa, aumentando gradualmente o limite de profundidade a cada

iteração, até que uma solução seja encontrada ou até que o limite máximo de profundidade seja atingido.

Depois do desenvolvimento dos conceitos referidos anteriormente, desenvolveu-se as classes do *package* larg que se encontra dentro do *package* PEE:

- `FronteiraFIFO` - esta classe herda da classe abstrata Fronteira, sendo que representa uma fronteira de exploração onde os seus elementos são inseridos e removidos segundo a lógica FIFO (First-In-First-Out). Ou seja, o primeiro nó inserido na fronteira será o primeiro a ser removido desta. Esta classe possui um método, `inserir()` insere um nó no fim da lista de nós da fronteira.
- `ProcuraLargura` - Esta classe é uma especialização de um mecanismo de procura (procura em grafo), sendo este o mecanismo de procura em largura (método de procura não informada). Este tipo de mecanismo consiste em explorar os nós mais antigos primeiro, ou seja explora primeiro todos os nós de uma certa profundidade para depois passar para os nós da profundidade seguinte. Este mecanismo consiste nos seguintes passos:

  1. Começa-se com o nó inicial e este é colocado na fronteira de exploração.
  2. Exploração em largura: Até se encontrar um estado do nó que corresponde ao estado objetivo, repete-se os seguintes passos:
  - Remove-se o nó mais antigo da fronteira (primeiro elemento).
  - Com a aplicação de operadores sobre o estado do nó removido, gera-se os seus nós sucessores.
  - Adiciona-se esses nós sucessores à fronteira (os nós mais recentes são colocados no fim da fronteira de exploração).
  3. Verificação do objetivo: Durante a exploração, sempre que um nó é removido da fronteira, verifica-se se o estado de este corresponde ao estado objetivo. Se for, a procura termina.

  Esta classe possui um construtor onde se instância uma `FronteiraFIFO()` e passa-se esta fronteira no `super()`.

Depois do desenvolvimento dos conceitos referidos anteriormente, desenvolveu-se as classes do *package* melhor_prim (este *package* tem como apoio o *package* aval sendo que este vai ser apresentado depois deste) que se encontra dentro do *package* PEE:

- `FronteiraPrioridade` - esta classe herda da classe abstrata Fronteira, sendo que representa uma fronteira de exploração onde os seus elementos são inseridos e removidos segundo um avaliador de prioridade dos nós. Ou seja, os nós nesta fronteira de exploração são listados segundo a avaliação fornecida (variável privada, __avaliador), quando um nó é inserido na fronteira, é colocado na lista segundo uma ordem crescente, para isto utiliza-se

o método `heappush`, que permite inserir um nó na fronteira. Para remover nós da fronteira de exploração utiliza-se o método `heappop` que segundo uma lista, remove e retorna o elemento mais pequeno (neste caso, segundo a sua prioridade). Em ambos os métodos depois de serem inseridos ou removidos os nós, estes métodos referidos anteriormente ajustam a ordem da lista segundo a sua prioridade mantendo a sua estrutura. Esta classe possui um construtor onde se usa o `super()` da classe fronteira e guarda-se o avaliador da prioridade dos nós num atributo privado. Esta classe também tem 2 métodos o `inserir()` e o `remover()`, tal como referi anteriormente o `heappush` é usado no inserir e o `heappop` no `remover().`

- `ProcuraMelhorPrim` - esta classe é uma especialização do mecanismo de procura em profundidade grafos, representando o mecanismo de procura melhor-primeiro. Este mecanismo, obtém uma solução óptima segundo o valor do seu custo, onde verifica se um nó gerado já foi explorado (pertence aos nós abertos ou fechados), caso este já tenha sido explorado, verifica-se o custo deste nó é menor que o custo do nó existente, se sim este é guardado na fronteira de exploração. Isto permite ignorar a exploração de nós que já existem no dicionário, caso o custo deste nó gerado for maior que o custo do nó com o mesmo estado. Esta classe possui um construtor onde se usa o `super()` para se passar uma instância da FronteiraPrioridade(avaliador), sendo o avaliador o argumento do construtor (guardado numa variável protegida). Esta classe possui o método `manter()` `que` indica se um nó deve ser guardado ou descartado, utiliza-se o super() para verificar se o nó pertence ao nós explorados e verifica-se o custo do nó gerado é menor que o custo do nó explorado com o mesmo estado, retorna `true` se o nó não existe no dicionário ou se o seu custo é menor que o nó explorado e `false` se já existe.
- `ProcuraCustoUnif` - esta classe é uma especialização do mecanismo de procura melhor-primeiro, representa o mecanismo de procura de custo uniforme. Neste mecanismo o valor da função de avaliação do custo dos nós é feita segundo o custo acumulado destes.

  Neste mecanismo de procura, a fronteira de exploração é ordenada pelo custo acumulado de cada nó, o nó com custo mais baixo vai ser expandido primeiro, pois este representa a melhor opção em termos de custo (no momento).

  Esta classe possui um construtor onde se usa o `super()` da classe ProcuraMelhorPrim para guardar a instância do avaliador especializado para este tipo de procura.

- `Heuristica` - classe abstrata que representa a função heurística, designada por **h(n)**, que consiste no custo estimado de um nó até ao estado objetivo, sem que o percurso até este tenha sido explorado. Para isto é necessário existir conhecimento acerca do domínio do problema, para assim ser possível orientar a procura para o objetivo desejado. Esta função **h(n)** necessita apenas de conhecer o estado do nó e o estado objetivo do problema. Esta

classe possui um método abstrato `h()` que quando implementado irá obter o custo estimado de um nó até ao estado objetivo do problema.

- `ProcuraInformada` - esta classe abstrata é uma especialização do mecanismo de procura melhor-primeiro, ela tem apenas métodos de procura informada, ou seja tem os mecanismos de procura que utilizam informações sobre o problema para assim orientar a sua exploração do espaço de estados em direção a uma solução de uma forma mais rápida que os métodos de procura não informada. Estes mecanismos ordenam a fronteira de exploração para assim os nós mais promissores (custo estimado até ao objetivo for menor) serem explorados primeiro. Essas informações sobre o problema, são obtidas através de uma função heurística que estima o custo de um nó em relação ao objetivo do problema. Esta classe possui um método `procurar()` que procura uma solução segundo um problema, definindo a heuristica do avaliador (ambos o problema e heuristica sao argumentos deste método).
- `ProcuraSofrega` - esta classe é uma especialização do mecanismo de procura informada, representando o mecanismo de procura Sôfrega. Este mecanismo baseia-se na seleção dos nós para a expansão da fronteira, onde se expande primeiro aqueles com menores valores **f(n)**, função de avaliação, que neste método consiste apenas no custo estimado do nó até ao estado objetivo, **h(n)**. Como **f(n)** é apenas igual a **h(n)**, não tem em conta o custo acumulado do nó, este mecanismo interessa-se apenas em diminuir o custo estimado para atingir o objetivo do problema. No entanto como este mecanismo não tem em conta o custo do percurso explorado, este pode levar a soluções sub-óptimas. Esta classe tem um construtor onde é instanciado um avaliador especializado para este tipo de procura, sendo passado no `super()` da classe para ser guardado na variável protegida `_avaliador`.
- `ProcuraAA` - Esta classe é uma especialização do mecanismo de procura informada, representando o mecanismo de procura A*.

  Este mecanismo baseia-se na seleção dos nós para a expansão da fronteira, onde se expande primeiro aqueles com menores valores **f(n)**, função de avaliação, que neste método consiste na soma do custo acumulado até ao nó, **g(n)**, e o custo estimado do nó até ao estado objetivo, **h(n)**. Esta classe tem um construtor onde é instanciado um avaliador especializado para este tipo de procura, sendo passado no `super()` da classe para ser guardado na variável protegida `_avaliador`.

Depois do desenvolvimento das classes do *package* melhor_prim desenvolveu-se as classes de apoio as várias procuras, *package* aval que se encontra dentro do *package* melhor_prim:

- `Avaliador` - classe abstrata que permite avaliar um nó, esta avaliação consiste na avaliação da sua prioridade segundo o seu custo, esta avaliação depende do mecanismo de procura escolhido. Uma função que avalia o custo de uma solução segundo um certo nó,

designa-se função **f(n)**. Quando maior for o seu valor, pior será a solução. Esta classe tem um método abstrato `prioridade()` que quando for implementado ele vai ser capaz de definir o segundo o custo de um nó a sua prioridade.

- `AvaliadorCustoUnif` - Esta classe herda da classe abstrata Avaliador, sendo que representa um avaliador de nós, onde a avaliação da sua prioridade é segundo o custo acumulado do nó (o custo do percurso deste o estado inicial até ao estado do nó). No mecanismo de procura de custo uniforme, a função **f(n)** é igual ao custo acumulado de um nó, representado por **g(n)**. Ou seja, nesta variante do método de procura melhor-primeiro **f(n) = g(n)**. Esta classe tem um método `prioridade()` que segundo o custo acumulado de um nó obtém-se a prioridade deste.
- `AvaliadorHeur` - Esta classe herda da classe abstrata Avaliador, sendo que representa um avaliador de nós, que permite utilizar heurísticas para definir a prioridade de um nó. Esta classe tem um método que `definir_heuristica()` que define um atributo protegido desta classe `_heuristica`.
- `AvaliadorSof` - Esta classe herda da classe abstrata AvaliadorHeur, sendo que representa um avaliador de nós, onde a avaliação da sua prioridade é segundo o custo estimado do nó até ao estado objetivo (estimativa do custo a partir do estado do nó até ao estado objetivo, sem que o percurso até este tenha sido explorado). No mecanismo de procura sôfrega, a função **f(n)** é igual ao o custo estimado do nó até ao estado objetivo, representado por **h(n)**. Ou seja, nesta variante do método de procura melhor-primeiro **f(n) = h(n)**. Esta classe tem um método `prioridade()` que segundo o custo estimado de um nó até ao estado objetivo obtém-se a prioridade deste.
- `AvaliadorAA` - Esta classe herda da classe abstrata AvaliadorHeur, sendo que representa um avaliador de nós, onde a avaliação da sua prioridade é segundo o custo acumulado do nó e segundo o custo estimado do nó até ao estado objetivo, custo mínimo (heurística admissível). No mecanismo de procura A*, a função **f(n)** é igual a soma do custo acumulado de um nó, **g(n)**, e o seu custo mínimo até ao objetivo, **h(n)**. Ou seja, nesta variante do método de procura melhor-primeiro **f(n) = g(n) + h(n)** (minimização do custo global). Esta classe tem um método `prioridade()` que segundo a minimização do custo global de um nó, obtém-se a prioridade deste.

Depois de implementar os conceitos teóricos sobre as PEE, criou-se (fora do package lib) o package contagem, dentro deste package vão ser implementadas especializações dos conceitos abordados para assim criarmos um pequeno problema de contagem para testar as bibliotecas criadas (segundo um valor inicial de 0 e um final de 9, quais incrementos usar para atingir o valor final: [1,

2, -1]). Dentro do package contagem criou-se o package mod_prob , para assim implementarmos especializações dos conceitos que envolvem o modelo do problema: estado, operador, problema e heurística (para testar as procuras informadas).

`EstadoContagem` - esta classe representa uma especialização de um Estado, representa uma configuração de um problema que neste caso é o valor da contagem. No construtor define um atributo privado valor (valor da contagem). Esta classe tem um propriedade para o valor e um método `id_valor()` que identifica o estado segundo o valor da contagem.

`OperadorIncremento` - Esta classe representa uma especialização de um Operador, representa a ação que permite a alteração de um estado que neste caso é o valor da incremento da contagem. No construtor define um atributo privado incremento (valor do incremento da contagem). Esta classe tem um propriedade para o incremento e tem dois métodos `custo()` que calcula o valor do custo (incremento**2) e o método aplicar, que aplica o operador ao estado transformando-o noutro, neste caso da contagem isto apenas se traduz em instanciar um novo estado com a soma do valor do estado recebido com o incremento do operador.

`ProblemaContagem` - Esta classe representa uma especialização de um Problema, sendo que neste caso é um problema de contagem. No construtor define um atributo privado estado inicial que representa um estado com o inicio da contagem, uma lista de operadores que são instanciados segundo uma lista de incrementos e um atributo valor final que indica o objetivo a atingir. Esta classe tem um método objectivo que indica se o objetivo do problema já foi ou não alcançado.

`HeuristicaContagem` - Esta classe representa uma especialização de uma Heuristica, representa a função que permite obter uma estimativa do custo estimado de um nó até ao estado objetivo, no problema da contagem, esta função **h(n)** permite obter a diferença absoluta entre o estado objetivo (__valor_final atributo privado da classe) e o valor da contagem (propriedade valor do estado recebido como parâmetro da função h).

5.3 - Teste da aplicação

Depois da realizar a implementação do problema, testou-se a aplicação para comprovar o seu bom funcionamento. Testou-se uma procura de espaço de estados não informada (em profundidade , removeu-se o -1 dos incrementos, pois isso levaria a um ciclo infinito, o que a variante “normal” deste método não se consegue tratar) e uma informada (sôfrega).

```
Procura em Profundidade
<mod_prob.estado_contagem.EstadoContagem object at 0x000001FB0C0E3880> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001FB0C4F96F0> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001FB0C4F9870> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001FB0C4F99F0> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001FB0C4F9B70> 2
```

Figura 18 - Teste da procura em profundidade

```
Procura Sofrega
<mod_prob.estado_contagem.EstadoContagem object at 0x000001EFF37E3850> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001EFF3E65840> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001EFF3E65A80> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001EFF3E65CC0> 2
<mod_prob.estado_contagem.EstadoContagem object at 0x000001EFF3E65E40> 1
```

Figura 19 - Teste da procura sôfrega

6. Projecto - Parte 4

6.1 - Modelo do problema

A quarta parte deste projeto, teve com o objetivo inicialmente desenvolver um agente deliberativo para implementar os métodos de PEE usados na parte 3 do projeto, de seguida desenvolveu-se uma agente deliberativo implementando Processo de Decisão de Markov. O objetivo em ambas as implementações do agente, eram a mesma do agente reativo da parte 2 , explorar o ambiente recolhendo os alvos que encontra e evitar os obstáculos no seu caminho. A diferença entre estas 3 partes finais é que o agente deixa de ser reativo e sim deliberativo, em vez de responder a estímulos do ambiente com ações este agente vai possuir um mecanismo de raciocínio que segundo um mecanismo de procura e de tomada de decisão o irá permitir deslocar até aos seus alvos e evitar obstáculos.

6.2 - Desenvolvimento e implementação do sistema

Ao longo do desenvolvimento deste projeto, podemos concluir que a cada parte a complexidade do nosso projeto tem vindo a aumentar, por isso iremos continuar as boas práticas que referimos nas restantes partes. Iremos começar por desenvolver um agente deliberativo, para depois desenvolvermos a integração deste agentes com as PEE e depois com processos de decisão sequencial, mais especificamente Processos de Decisão de Markov. Este projeto irá continuar a ser desenvolvido no package iasa_agente, sendo que dentro deste vamos inicialmente trabalhar no *package* agente, e quando integrarmos estes aos métodos referidos anteriormente iremos trabalhar no *package* que envolve as abstrações dos conceitos teóricos, o package lib (bibliotecas).

Para o desenvolvimento dos conceitos referidos no *package* agente, começou-se por criar um *package* para o controlo deliberativo (controlo_delib) por desenvolver as classes `ControloDelib`, `MecDelib`, depois dentro de um *package* modelo dentro do controlo_delib desenvolveu-se as classes `EstadoAgente`, `ModeloMundo`, `OperadorMover`:

- `EstadoAgente` - esta classe herda da classe abstrata Estado, sendo que representa uma configuração de um problema, sendo esta a posição do agente deliberativo no ambiente virtual (mundo). Esta posição consiste nas coordenadas do agente no mundo. Para representar a localização do agente, utiliza-se um tuplo com as suas coordenadas em x e y

(classe `Posicao` fornecida pelos docentes no package SAE). Esta classe tem um construtor onde inicializa um atributo privado, `__posicao` que representa a posição do agente no ambiente virtual e também guarda-se num atributo privado `__id_valor` o `hash` da posição . Esta classe também tem uma propriedade que permite a leitura da posição do agente e um método `id_valor()` que permite identificar o estado segundo um id único o hash da posição (atributo, `__id_valor`).

- `OperadorMover` - Esta classe herda da classe abstrata Operador, sendo que representa uma ação que permite a alteração de um estado, sendo esta a deslocação do agente no mundo. Como um estado consiste na posição do agente deliberativo no ambiente, o operador vai ser o que permite a transição entre estados, logo os operadores consistem na deslocação do agente no mundo. Para isto é necessário conhecer o mundo, para se conhecer os vários estados possíveis e é necessário saber a direção da deslocação do agente. Esta classe tem um construtor onde se instância una Accao segundo a direção recebida como parâmetro (direção do movimento do agente), inicializa-se um atributo privado que guarda referência a um modelo do mundo, ou seja uma representação interna do ambiente. Esta classe possui duas propriedades, uma para permitir a leitura da ação e outra para permitir a leitura do ângulo da direção da ação. Esta classe também tem 3 métodos: `aplicar()` que aplica o operador ao estado atual do agente, permitindo a deslocação do agente, caso o novo estado criado, seja uma das configurações possíveis do mundo; `custo()` indica o custo da transição entre estados, através do cálculo da distância entre os estados recebidos (estado atual do agente, e o futuro estado deste, após a aplicação do operador); `__translacao()` método privado utilizado no `aplicar()`, que realiza a translação da posição do agente com base na distância e ângulo fornecidos. Utiliza-se a modularização, para assim criarmos um método que apenas faz a translação geométrica, sem se preocupar com o contexto do problema onde este se encontra.

- `ModeloMundo` - Esta classe herda da classe abstrata ModeloPlan, sendo que representa o modelo do mundo, ou seja, uma representação interna do ambiente, sendo esta criada e atualizada segundo as perceções do agente. Nestes agentes deliberativos utiliza-se memória para guardar-se informações sobre o ambiente como: os elementos, que abrange todos os constituintes percecionados pelo agente (como alvos e obstáculos); o estado atual do agente; todos os estados ou seja configurações possíveis do mundo;
  Os operadores que consistem em ações que permitem que o estado do agente seja alterado, sendo que isto leva á sua deslocação no mundo.
  Esta representação do mundo é essencial para um agente deliberativo, pois ela permite a compreensão e interação do agente com o mundo. Esta classe tem um construtor onde se

inicializa a lista de operadores, segundo instâncias de OperadorMover, onde se passa o modelo do Mundo (através do `self`) e uma direção sendo que esta representa uma das direções do `enumerator Direccao` (direções em que o agente consegue se mover). Esta classe possui duas propriedades, uma que permite a leitura da alteração do modelo de mundo e uma que permite a leitura de elementos percecionados pelo agente. Esta classe também tem os seguintes métodos: `obter_estado()` permite obter o estado atual do agente; `obter_estados()` permite obter a lista de todas as configurações possíveis do mundo; `obter_operadores()` permite obter a lista de operadores; `obter_elemento()` permite obter um elemento da lista de elementos, segundo a posição do estado recebido; `atualizar()` atualiza o modelo do mundo segundo a perceção do agente. Inicialmente atualiza-se o estado atual do agente, segundo a posição da perceção, de seguida atualiza-se a lista dos estados possíveis segundo a perceção do agente e atualiza-se os elementos percecionados. Por fim atualiza-se a variável privada `__recolha`, para assim identificar-se a atualização ou não do modelo do mundo. Esta variável é atualizada segundo a variável recolha da percepção, esta é alterada caso o agente tenha recolhido um alvo, o que significa que o modelo do mundo foi alterado; `distancia()` calcula a distância entre o estado atual do agente e o estado fornecido; `mostrar()` utilizado para a visualização gráfica do ambiente virtual.

- `MecDelib` - Esta classe representa um dos componentes do raciocínio prático, sendo este o Raciocínio sobre Fins (Deliberação). Este mecanismo deliberativo consiste em analisar o modelo do mundo e determinar segundo os elementos percecionados pelo agente, quais destes devem ser considerados objetivos (no nosso agente estes objetivos a realizar são os alvos que o agente tem de recolher). Esta classe tem um construtor onde se guarda num atributo privado uma representação interna do ambiente. Esta classe tem um método deliberar() que consiste na decisão de quais dos elementos existentes no modelo do mundo, são os objetivos do agente. Inicialmente obtemos os vários estados possíveis do modelo do mundo, para depois percorrermos a lista destes, e testar se o elemento nessa posição do mundo é do tipo alvo, se sim guardamos o estado com esse elemento na lista dos objetivos para assim guardar-se a posição dos vários alvos.
- `ControloDelib` - Esta classe herda da classe abstrata Controlo, sendo que representa um controlo deliberativo de um agente. Estes agentes tomam decisões segundo uma representação interna do ambiente, o modelo do mundo, e segundo os seus objetivos. Estes objetivos e o plano como estes vão ser concretizados são definidos segundo um raciocínio orientado para ações, também designado por raciocínio prático. Este raciocínio é composto por duas componentes:

-Raciocínio sobre fins, designado por deliberação, que segundo a perceção do agente tem de analisar os elementos que se encontram no ambiente/mundo e escolher aqueles que

representam os objetivos a atingir.

-Raciocínio sobre meios, designado por planeamento, que segundo os vários objetivos tem de planear um plano de ações para assim conseguir atingir o resultado esperado.

Esta classe tem um construtor onde se instância o modelo do mundo e um mecanismo deliberativo, pois este controlo necessita de uma representação do mundo para compreender e interagir com ele, para além disso ele também guarda um atributo privado, o planeador de um agente. Esta classe tem os seguintes métodos: `processar()` processa as perceções do agente e decide qual ação tomar. Este método segue o processo de tomada de decisão e ação, onde inicialmente recebe como parâmetro o que o agente observa no ambiente, atualiza o modelo do mundo, segundo essa perceção, e caso haja uma alteração no modelo do mundo, gera-se um conjunto de objetivos (deliberação) e gera-se um plano de ação (planeamento). Por fim executa-se o plano de ação, sendo que o agente irá executar a ação planeada pelo raciocínio; `__assimilar()` atualiza o modelo do mundo com a perceção do agente; `__reconsiderar()` verifica se devemos reconsiderar o plano de ações; `__deliberar()` atualiza a lista de objetivos do agente; `__planear()` planea as ações e atualiza o plano de ação; `__mostrar()` utilizado para a visualização gráfica do ambiente virtual; `__executar()` se houver um plano definido, ele obtém o operador de um passo da solução segundo o estado atual do agente. Caso o plano e o agente estejam assíncrono (operador retornado pelo método `obter_accao` do plano é `None`), coloca-se o plano a `None`, pois este está desatualizado em relação ao agente;

Depois do desenvolvimento das classes do *package* controlo_delib desenvolveu-se as classes de apoio aos agentes deliberativos, criou-se o *package* plan que se encontra dentro do *package* lib, sendo que este vai ser composto por outros *packages* e por duas classes abstratas, `Planeador` e `Plano`, dentro de um package modelo também se encontra outra classe abstrata `ModeloPlan`:

- `Planeador` - classe abstrata que representa um planeador de um agente deliberativo, ou seja o mecanismo que permite decidir a sequência de ações a executar de forma a realizar todos os objetivos deliberados. Esta classe tem um método abstrato `planear()` , este método irá realizar o planeamento das ações, para se obter o plano da ação.
- `Plano` - classe abstrata que representa um plano de ações de um agente deliberativo, ou seja um plano consiste no resultado de um planeador, uma sequência de ações que o agente deve executar para atingir os seus objetivos. Esta classe tem um método abstrato `obter_accao()` , este método irá permitir obter um operador segundo um estado.
- `ModeloPlan` - classe abstrata que representa um modelo de planeamento de um agente deliberativo. Estes modelos quando implementados irão criar representações internas do ambiente, onde irão guardar informações sobre o mundo, como o estado atual do agente, os vários estados possíveis do mundo e os vários operadores que permitem a transição entre estados. Esta classe tem três métodos abstratos `obter_estado()`, este método irá

permitir obter o estado atual do agente, `obter_estados()`, este método irá permitir obter a lista de estados possíveis no mundo, `obter_operadores()`, este método irá permitir obter a lista de operadores.

Depois do desenvolvimento das classes anteriores, criou-se o *package* plan_pee que se encontra dentro do *package* plan, sendo que este vai ser composto por um *package* mod_prob, que tem duas classes `HeurDist` e `ProblemaPlan`, dentro do *package* plan_pee existe duas classes `PlaneadorPEE` e `PlanoPEE.`

- `HeurDist` - classe representa uma especialização de uma Heuristica, representa a função que permite obter uma estimativa do custo de um nó até ao estado final. Esta classe tem um construtor, onde se guarda o valor do objetivo do problema. Esta classe tem um método `h()` que obtém o custo estimado do nó até ao estado final do problema, consiste no calculo da distância euclidiana entre a posição do estado recebido e a posição do estado final.
- `ProblemaPlan` - classe representa uma especialização de um Problema, sendo que representa um problema de um planeador de um agente deliberativo. Esta classe tem um construtor, onde se utiliza o `super()` para guardar o estado inicial do problema e os seus operadores. Esta classe tem um método `objectivo()` que compara o estado que recebe como parâmetro com o estado final do problema (objetivo), para assim saber-se se o objetivo já foi ou não atingido
- `PlaneadorPEE` - classe herda da classe abstrata Planeador, sendo que representa um planeador baseado no mecanismo de raciocínio automático, procura em espaço de estados. Esta classe tem um construtor onde se instância um mecanismo de procura em espaço de estados, sendo este o mecanismo de procura informada, A*. Esta classe tem um método sendo este `planear()` este método permite a realização do planeamento das ações, para se obter o plano de ação. Considera-se que a lista de objetivos vem ordenada segundo a sua prioridade. Logo escolhe-se o primeiro elemento desta lista. Cria-se uma instância de `ProblemaPlan` e de `HeurDist`, pois no mecanismos de procura A* é necessário passar um problema e uma heuristica, para assim obter-se uma solução ao problema, sendo que caso esta exista instância-se um plano sobre ela.
- `PlanoPEE` - classe herda da classe abstrata Plano, sendo que representa um plano de ações para uma procura de espaço de estados. Esta classe tem um construtor onde se guarda numa variável privada `__passos`, os vários passos da solução do plano, sendo isto possível, pois esta solução é iterável segundo os passos desta. Esta classe tem dois métodos `obter_accao()`, caso uma solução exista, permite obter um operador de um passo da solução. Se a solução existir, remove-se o primeiro elemento da lista `__passos` e caso o estado deste for igual ao estado recebido retorna-se o operador do passo da solução; `mostrar()` utilizado para a visualização gráfica no ambiente virtual.

Depois do desenvolvimento das classes anteriores, já se desenvolveu todos os passos necessários para criar uma agente deliberativo capaz de utilizar mecanismos de procura em espaço de estados, para navegar no Simulador, no ambiente virtual, tal como o agente reativo fazia (explorava o ambiente, recolhia os alvos e evitava obstáculos).

Para criar um agente deliberativo capaz de explorar o ambiente, recolher os alvos e evitar obstáculos, segundo o mecanismo Processo de Decisão de Markov, ainda é necessário desenvolver código, para este caso criou-se um *package* pdm que se encontra dentro do *package* lib. Este *package* é composto por 3 classes, sendo que uma delas está dentro de um *package* modelo (classe `ModeloPDM`) as outras duas classes são `MecUtil` e `PDM`.

- `ModeloPDM` - Classe abstrata que representa um modelo do processo de decisão de Markov. Com este modelo é possível representar o mundo através dos seguintes componentes:

  - S -> Conjunto de estados do mundo, ou seja conjunto de componentes que definem o mundo, como as suas posições e configurações. O método abstrato `S()` quando implementado irá permite obter uma lista de estados.

  - A(s) -> Conjunto das ações possíveis que podem ser tomadas num certo estado, s. Ou seja as ações que um agente pode tomar num certo estado. O método abstrato `A()` permite obter uma lista de operadores possíveis segundo um estado.

  - T(s, a, sn) -> Função de transição de estado, esta função define a probabilidade de transição de um estado s para um estado sn, através de uma ação a. O método abstrato `T()` permite obter a probabilidade de transição de um estado para outro, através de um operador.

  - R(s, a, sn) -> Função de recompensa, esta função determina a recompensa ao transitar de um estado s para um estado sn através de uma ação a. A recompensa representa o ganho ou perda num determinado estado, s. O método abstrato `R()` permite obter a recompensa de transição de um estado para o estado sucessor, através de um operador.

  Nesta classe não é definido um componente que representa a taxa de desconto (representado por um gama). Esta taxa determina a importância das recompensas futuras em comparação com as recompensas imediatas. Esta classe tem mais um método abstrato `suc()` que permite obter a lista de estados sucessores para um certo estado, segundo um operador.

- `MecUtil` - Esta classe representa um dos componentes fundamentais dos processos de decisão de Markov, a utilidade. A utilidade, representada por U, consiste na recompensa de um agente ao executar uma ação num estado, para assim transitar para outro. Esta é diferente da função R da recompensa, pois a utilidade é a soma das recompensas "atuais" (segundo o estado atual do agente) e as recompensas futuras esperadas, ponderadas pela taxa de desconto, gama (Recompensa descontadas no tempo). O método construtor desta

classe recebe um modelo PDM, um valor de taxa de desconto e um delta *max* (usado no cálculo da utilidade), cada um destes valores é guardado no seu respetivo atributo privado. Esta classe tem dois métodos: `utilidade()` que calcula o valor da utilidade de estado. Inicialmente inicializa-se um dicionário onde cada estado tem o seu valor a 0 (valor da utilidade). De seguida utiliza-se um *loop* `while True`, onde se guarda um cópia do dicionário e depois entra-se num ciclo `for` onde se vai iterar segundo os estados da lista de estados possíveis. Neste `for` atualiza-se os valores da utilidade para cada estado, segundo o valor máximo obtido no cálculo da utilidade para cada operador. Depois de ciclo for verifica-se se a condição de paragem do ciclo `while` foi atingida, esta consiste no valor da diferença da utilidade da cópia do dicionário e o modificado. Se este valor for inferior ou igual ao `delta_max`, o ciclo para e retorna-se o dicionário com as utilidades calculadas para cada estado; `util_accao()` calcula a utilidade de uma ação. Este método consiste no somatório de uma fórmula para todos os estados sucessores de um estado quando lhe é aplicado um operador.

$$\sum_{s'} T(s,a,s')\left[R(s,a,s')+\gamma U(s')\right],\ \forall s \in S$$

Figura 20 - Morgado, Luís. (2024). Fórmula utilizada no método util_accao()

- `PDM` - Classe que representa o processo de decisão de Markov, PDM. Este processo é um processo de decisão sequencial, onde os estados são afetados por ações, utiliza-se funções da probabilidade de transição de estados e de recompensa, para que o agente consiga tomar as melhores decisões no ambiente. Um PDM consiste num processo de decisão sequencial com a propriedade de Markov, esta propriedade diz que o futuro é independente do passado, ou seja para prever os estados sucessores do estado atual de um agente apenas importa esse estado e não os seus valores anteriores. Um dos componentes fundamentais dos processos de decisão de Markov é a política comportamental, nesta classe foi implementada uma política comportamental não determinista, neste política é feita uma distribuição de probabilidade sobre as possíveis ações para cada estado. O método construtor desta classe recebe um modelo PDM e guarda-o num atributo privado e instância um `MecUtil` que é guardado num atributo privado também. Esta classe tem dois métodos: `politica()` retorna o dicionário da política óptima para o conjunto de estados possíveis com base no valor máximo do calculo da utilidade das ações possíveis para cada estado. Inicialmente inicializa-se um dicionário vazio, percorre-se a lista de estados possíveis e caso este estado tenha uma lista de ações, faz-se o cálculo da utilidade para cada ação, para depois obter-se o seu valor máximo, sendo que este é guardado no dicionário; `resolver()` resolve o processo de decisão de Markov, calculando as utilidades e a política óptima para o conjunto de estados possíveis.

Tal como se fez para o PEE, para o mecanismo PDM criou-se o *package* plan_pdm que se encontra dentro do *package* plan, sendo que este vai ser composto por um *package* modelo, que tem uma classe `ModeloPDMPlan`, dentro do *package* plan_pdm existe duas classes `PlaneadorPDM` e `PlanoPDM`.

- `PlaneadorPDM` - Esta classe herda da classe abstrata Planeador, sendo que representa um planeador automático com base em processos de decisão de Markov, PDM. Este planeador possui um atributo privado `__gama`, que num processo de decisão de Markov, consiste numa taxa de desconto em relação ao tempo. A taxa de desconto determina a importância das recompensas futuras em comparação com as recompensas imediatas. Esta classe tem um construtor onde se guarda em atributos privados o valor de gama e o de delta *max.* Esta classe tem um método sendo este `planear()` este método permite a realização do planeamento das ações, para se obter o plano de ação. Inicialmente verifica-se se os objetivos existem, caso existam instância-se o `ModeloPDMPlan` e `PDM` (processo de decisão de Markov). De seguida utiliza-se o método resolver da classe `PDM`, que resolve o processo de decisão de Markov, retornando as utilidades e a política óptima para o conjunto de estados possíveis, estes valores depois são usados para instanciar um `PlanoPDM`.
- `PlanoPDM` - esta classe herda da classe abstrata Plano, sendo que representa um plano de ações para um PDM, processos de decisão de Markov. Esta classe tem um construtor onde se guarda numa variável privada a utilidade calculada para cada estado do mundo e a política, um dicionário da política de estados. Esta classe tem dois métodos `obter_accao()`, retorna um operador segundo o estado recebido, caso exista política; `mostrar()` utilizado para a visualização gráfica no ambiente virtual.
- `ModeloPDMPlan` - esta classe herda da classe abstrata ModeloPDM e ModeloPlan, sendo que representa um modelo de um problema que pode ser resolvido através de um processo de decisão de Markov. Utiliza-se fatorização sem herança para assim delegar ao modelo do plano, utilizando os seus métodos, para se obter o estado atual do agente, os estados do mundo e seus operadores. No construtor desde método pré calcula-se as transições, isto é utilizado quando o modelo do mundo o permite, que no nosso caso permite. Isto irá melhorar o desempenho do mesmo, pois assim no raciocínio do nosso agente não é necessário calcularmos as transições. Como o nosso modelo é determinista consegue-se saber o próximo estado de um agente depois da aplicação de uma ação, logo neste dicionário `__transicoes` associa-se apenas estados a operadores e não probabilidades.

6.3 - Teste da aplicação

Depois da realizar a implementação da arquitetura dos agentes deliberativos, testou-se o agente com PEE e depois com PDM para comprovar o seu bom funcionamento, em ambas as imagens do lado esquerdo temos o agente num simulador que recolhe alvos, do lado direito temos uma painel onde podemos ver setas amarelas que indicam o caminho que o agente vai percorrer, ou seja a trajetória da solução (PEE) e no caso da parte 4 as setas representam a direção da ação, sendo que neste simulador de PDM o cor verde representa a recompensa ou seja quanto menor a distância ao alvos maior será os ganhos. Quanto mais perto do objetivo mas claros eles fica e mais longe indica mais perdas logo a cor é mais escura.

No fim da simulação, o agente já recolheu todos os seus alvos portanto fica a parado no mapa, no local onde apanhou a ultimo alvo.Sendo que em ambos os casos obteve-se o comportamento esperado.

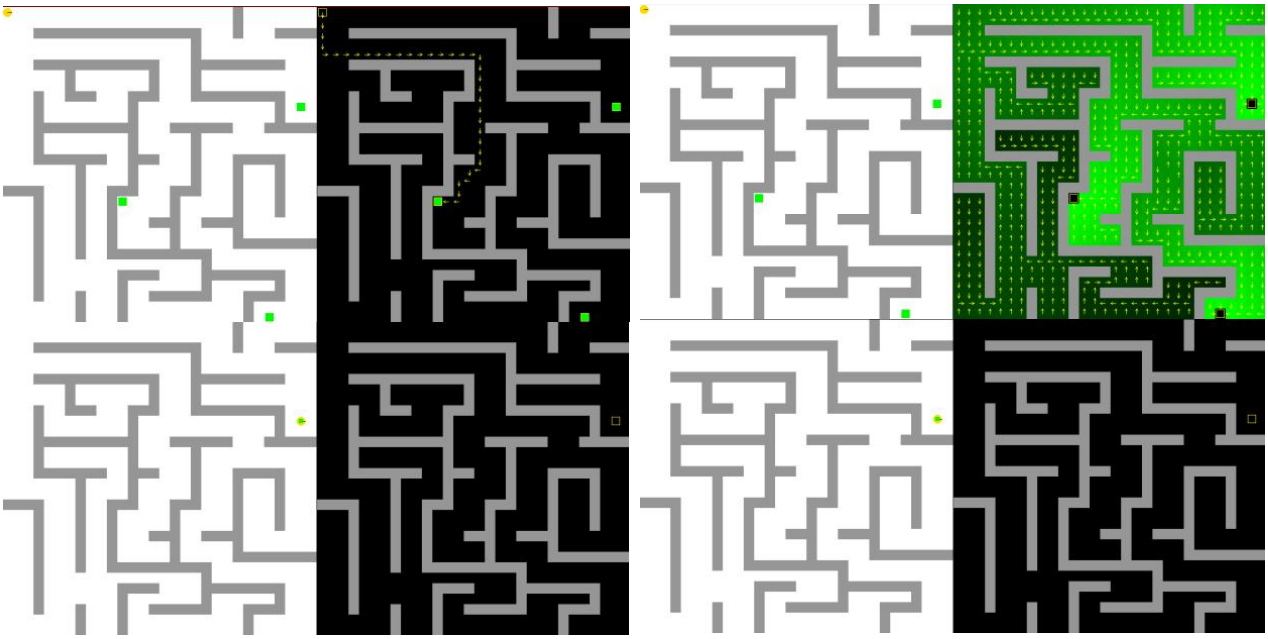


Figura 21 - Teste do Sistema (PEE esquerda, PDM direita)

7. Revisão do projecto realizado

O desenvolvimento do código em sala de aula, onde este era explicado pelo docente da cadeira e em alguns casos demonstrado pelo próprio, faz com que não aja muitos erros no código desenvolvido.

Algumas vezes durante as aulas ocorreram erros de *imports*, no entanto estes acabavam por ser corrigidos, pois muitas vezes esses erros ocorriam devido a distrações.

Em termos de código, o único erro que ocorreu numa entrega durante a aula, foi na classe `RespostaEvitar` do package evitar, onde eu no método `activar()` testava o contacto com um obstáculo segundo a direção inicial (que antigamente guardava como um atributo da classe) o que iria dar erros pois esta não é a direção certa. O teste deve ser feito segundo a direção atual da ação, sendo que esta depois é atualizada para assim permitir o agente ao percecionar obstáculos conseguir evitá-los. Este erro foi corrigido logo a seguir e documentado, pois essa entrega foi a meio da aula o que permitiu aos alunos que tinham erros no código, de revê-lo e corrigi-lo.

8. Conclusão

Ao longo deste projeto foi possível desenvolver código utilizando as boas práticas do desenvolvimento de software, onde se conseguiu ter quatro projetos com complexidade organizada, onde primeiro criava-se abstrações dos conceitos teóricos que envolviam os agentes reativos ou os agente deliberativos, para depois implementar estes mesmos conceitos dentro do contexto em que os sistemas estavam inseridos. Isto facilita bastante o trabalho não só dos programadores, mas também facilita o entendimento do mesmo para outros, que possam tentar alterá-lo ou expandi-lo.

Na parte 1 e 2 deste projeto, foram implementados agentes reativos, com isto ficaram bem salientadas os conceitos básicos que envolvem estes agentes, como os comportamentos e a coordenação para atingir comportamentos complexos de agentes.

Na parte 3 e 4 deste projeto, estudou-se mais algoritmos de inteligência artificial, com estes algoritmos foi possível criar agentes que através de representações internas do ambiente, onde se encontravam, conseguiam-se tornar autónomos e executar objetivos, sendo que isto é possível através de agentes deliberativos.

Bibliografia

Consulta e material de apoio:

Morgado, Luís. (2024). Inteligência Artificial para Sistemas Autónomos - Moodle